Phase 4 – Processes, signals and limits

Acme Analytics Linux Platform · all final files in one print copy · Muhammad Bilal Javed · 24 Oct – 4 Nov 2026
Version: 4 November 2026, 18:20 (Germany time) – Phase 4 COMPLETE – includes Step 4, the incident drill, GLPI ticket and L3 Problem
compact: A4 report pages are printed two per sheet

Part	What	Where
A	Steps 4.1–4.2 – slow app, runaway process, load average, signals, gw01-off incident, pipes	Day 7 report
B	Step 4.3 – signals, lock, trap, the archive queue system and worker, L1 → L2 escalation	Day 8 report
C	Step 4.4 – how processes are born: PID, fork, exec, wait, orphan, zombie (+ zombie essentials)	processes file + 1 page
D	Step 4.5 concept – cgroups, limits, cache and swap, Docker/Kubernetes, networking	2 diagrams
E	Where limits can be set (7 places) and where they are stored	2 pages
F	Foundations – disk, RAM, kernel, systemd, mounts · live test vs permanent	2-page overview
G	The 3 limit tests (KILL · SLOW · WAIT) · who the OOM killer chooses · OOM score	2-page revision sheet
H	Step 4 on stor01 – permanent limits for archive-worker + reboot proof *+ Step 4 explained: CPUQuota in time, NFS/ backup sharing, TasksMax vs restart limits*	Step 4 file + 3 diagrams
I	First incident drill on stor01 – "deleted the file, but the disk is still full" (lsof +L1) + GLPI ticket text	drill file
J	Closing Phase 4 – GLPI Incident ticket #4, L3 Problem behind ticket #3, snapshot 50-processes	GLPI section

Phase 4 in one paragraph: A process is a running program with a PID, born by fork and exec from its parent, and every one of them lives in a cgroup box that systemd creates. Signals let me stop or pause processes safely; nice decides who goes first, while limits on the box decide how much at most: MemoryMax kills (the kernel's OOM killer picks the highest score in that box), MemoryHigh slows the job into swap, CPUQuota makes it wait, TasksMax caps the number of processes. A test with systemd-run is a live change that disappears; a limit written into a file – set-property without --runtime – is read at every boot and stays, as the reboot test on stor01

proved. When df and du disagree, a deleted file is still held open – lsof +L1 finds the process, and restarting it frees the space.

Before Phase 4 (processes, signals, limits) – what you need from Phases 0–3

5-minute revision · left = what you already know · right = where it comes back in Phase 4

1	**boot chain and PID 1** Phase 0	• systemd is the first process • it starts all services • getty shows the login	→ how processes are born: systemd (PID 1) → getty → login → bash → my commands (fork, exec)
2	**services and unit files** Phase 3	• .service units, enable, restart • Restart=on-failure • drop-in folders .d/	→ the archive worker service (Nice=19) · one cgroup box per service · permanent limits as drop-ins
3	**timers and the nightly backup** Phase 3	• backup runs by itself at night • disk-full alert timer • catch-up after downtime	→ why limits matter: a big archive job must not slow the backup and NFS on stor01
4	**storage on stor01** Phase 2	• /backups LVM volume • NFS share to app01 • df -h	→ the archive queue lives on stor01 · the deleted-but-open file drill uses /backups
5	**SSH between servers** Phase 1	• key-only SSH • commands on another server • rsync over SSH	→ pipes across servers: tar | pv | ssh stor01 streams an archive without a temp file
6	**GLPI and escalation** Phase 1A	• tickets, L1 → L2 • requester vs item • solution + approval	→ ticket #3 escalation · Incident #4 for the drill · the first L3 Problem (root cause)

Final paragraph: Phase 4 looks inside the running system. systemd as PID 1 from Phase 0 is the start of every process family; the services and drop-ins from Phase 3 become the archive worker with its own cgroup box and permanent limits; the backup and NFS on stor01 from Phase 2 are what those limits protect; SSH from Phase 1 carries the tar | ssh pipe; and GLPI from Phase 1A records the escalation, the incident drill and the first L3 Problem.

Project 2 – Acme Analytics Linux Platform

Day 7 · Phase 4 starts: where a slow application comes from, stopping a runaway process, pipes

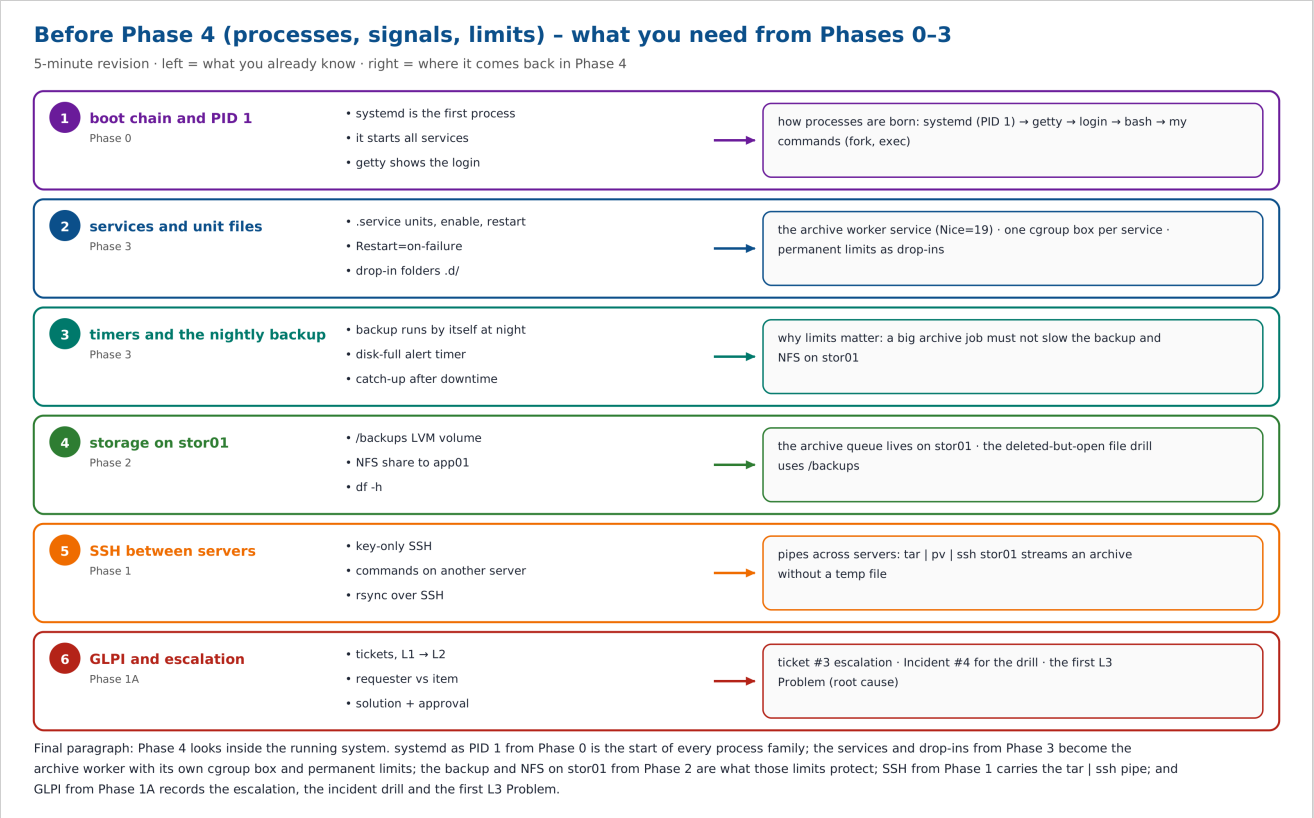
Muhammad Bilal Javed · 24 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	4 – Processes, signals and limits
Hosts worked on	app01, gw01 (powered on), svc01 (GLPI)
Status	Phase 3 ✅ (Day 6) · Phase 4 🟡 – Step 4.1 runaway process ✅ (concept, hands-on on app01, GLPI ticket #2) · Step 4.2 pipes ✅ (FIFO as reference)
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary. Previous day: Day 6.

Before Phase 4 – quick revision (Phases 0–3)

💡 **5 minutes:** what I already know from Phases 0–3 and where it comes back in Phase 4.



Before Phase 4 – quick revision

1. A question from industry – too many backups, and backup logs that grow too big

💡 **In plain words:** a friend who works as a Linux administrator asked: in a real company backups are taken all the time – what happens when the backups, and the backup **logs**, become huge? These are two different problems with two different fixes.

Problem	Industry fix	Already in this project
The backups grow	Retention (e.g. 7 daily, 4 weekly, 12 monthly) · incremental copies · deduplication (restic, borg) · move old copies to cheaper storage (AWS S3 → Glacier with a lifecycle rule)	`find … -mtime +7` keeps 7 days · `rsync --link-dest` hard links (Day 6)
The backup log grows	Log a summary, not every file (`rsync -v` writes one line per file) · rotate and compress logs with `logrotate` · journal size limit (`SystemMaxUse=`) · watch `/backups` and `/var/log` too	One `logger` line per run · logrotate comes in Phase 5

Interview line: *"I keep backups small with incremental copies, retention and cheaper storage for old copies, and I keep logs small by logging summaries and rotating them with logrotate."*

2. Real-world troubleshooting practice – how the incident drills fit in

💡 **In plain words:** to train like an on-call administrator, each phase will end with an **incident drill on my own Acme servers**: something realistic breaks, and I fix it like a real ticket – triage → hypotheses → commands → fix → verify → write-up in GLPI. The scenarios come from a hands-on troubleshooting lab (SadLab) and an L2/L3 incident workbook; only the cases that match this platform are used.

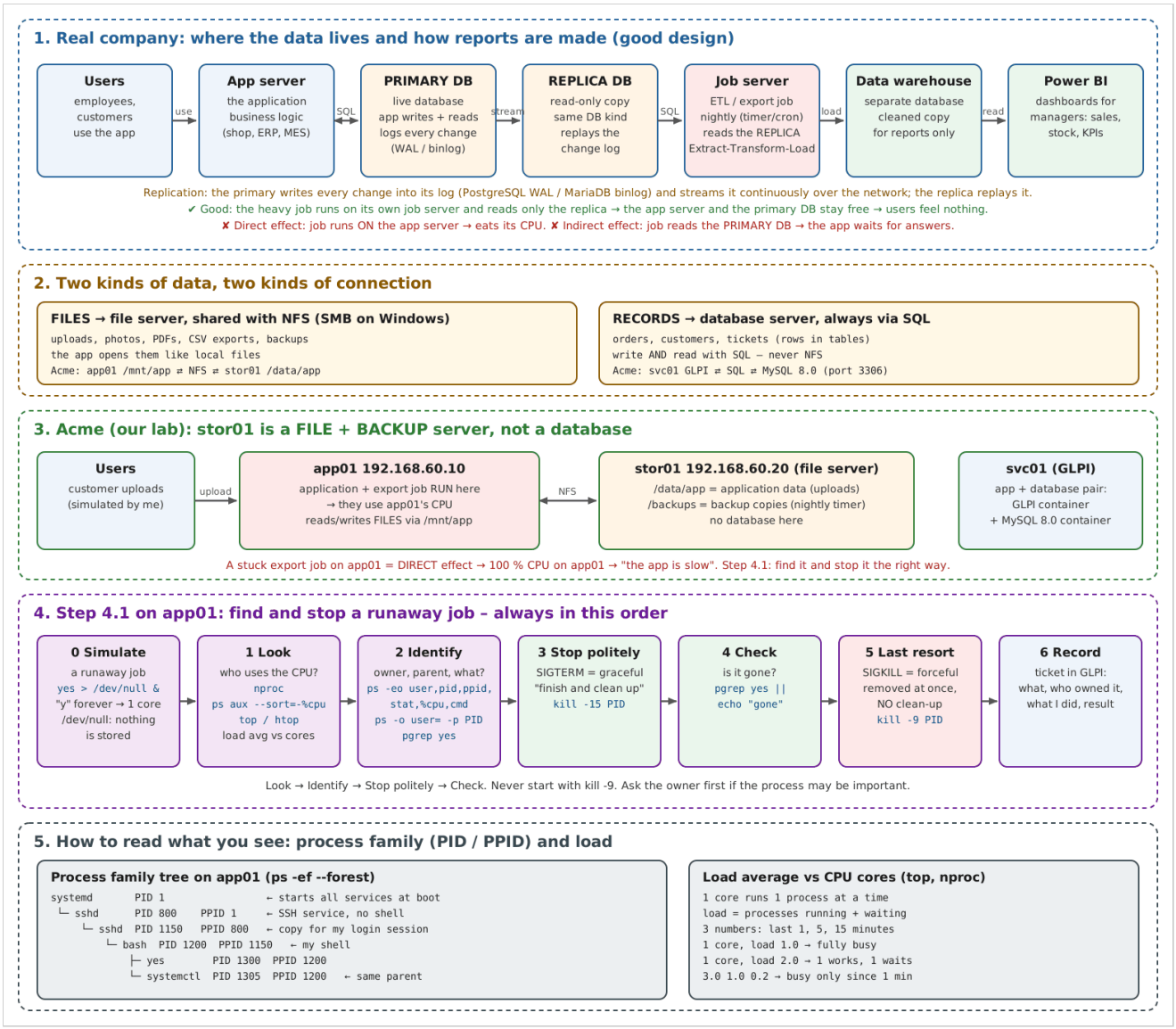
Phase	Drill
4 – Processes, signals	Runaway log writer · deleted file still filling the disk (`lsof +L1`) · "the server is slow"
5 – Logging	Backup log too big → logrotate · log rotation stalls a service
Backups	Backup "succeeds" but the restore fails → add a verify step to the nightly backup
6 – Security	SSH key login rejected · 403 Forbidden · firewall rule blocks a service
7 – Web service	Nginx won't start · works locally, not from outside
8 – Disaster drill	Read-only root after a power cut · wrong device mounted · inodes full

Several "advanced" workbook incidents were already solved for real in this project: a process hung on NFS after a storage failover (Day 5 stale mount and **D state**), an LVM snapshot during maintenance (Day 5), the wrong device mounted (why fstab uses **UUIDs**, Day 4), a backup that must be proven by a restore (Day 4 restore drill), and a cron job versus a timer (Day 6).

3. Phase 4 starts – Step 4.1: where a slow application comes from

💡 **The story:** app01 is Acme's application server. In real companies the most common daily problems are a **process that eats all the CPU**, a **job that hangs**, or a program that **won't stop properly**. Phase 4 teaches how to **find** the guilty process, **stop it the right way**, and finally **cap** what a service may use, so one bad program cannot freeze the whole server. Before touching commands, I worked out where such a process comes from in a real company and in Acme.

📸 **Architecture – where the data lives, how reports are made (primary → replica → job server → data warehouse → Power BI), files vs records, Acme, the order of work on app01, the process family and the load average:**



Step 4.1 architecture

Remember it in four lines:

```
Real company:  Users → App server ⇄ PRIMARY DB → (WAL/binlog stream) → REPLICA → Job server (ETL) → Data warehouse → Power BI
Replication:   the app writes SQL only to the primary; the primary logs every change and streams the log to the replica, which replays it
Acme:          Users → app01 (runs the app) ⇄ NFS (files) ⇄ stor01 (stores files + backups)  ·  svc01: GLPI ⇄ SQL ⇄ MySQL 8.0
Step 4.1:      Simulate → Look → Identify → Stop politely → Check → (kill -9 last) → Record
```

All on **app01**:

```
sudo apt install -y htop stress-ng
nproc                                                    # how many CPU cores
yes > /dev/null &                                        # simulate a runaway job
ps aux --sort=-%cpu | head -5                            # who uses most CPU
top                                                      # load average, %CPU, STAT — q to quit
htop                                                     # same, clearer — q to quit
ps -eo user,pid,ppid,stat,%cpu,cmd --sort=-%cpu | head -5   # owner, parent, state, command
ps -o user= -p $(pgrep yes)                              # who owns it
ps -ef --forest | less                                   # the process family tree
kill -15 $(pgrep yes)                                    # SIGTERM — graceful stop
pgrep yes || echo "gone"                                 # check
```

Term	Meaning
Runaway job	Busy and out of control – 100 % CPU, never finishes
Hung job	Stuck waiting – almost 0 % CPU, never finishes
Load average	Processes running + waiting for the CPU; 1-, 5- and 15-minute averages, compared with `nproc`
PID / PPID	The process's own number / the number of the process that started it
SIGTERM (`kill -15`) / SIGKILL (`kill -9`)	Graceful stop with clean-up / forceful stop, no clean-up – last resort
Primary / replica	Live database / read-only copy of the same kind, kept up to date by streaming the change log
WAL / binlog	The change log of PostgreSQL / MariaDB – the source of replication

3.1 The hands-on run on app01

Issue – `apt install` hangs at `0% [Working]`

Seen	On app01, `sudo apt install -y htop stress-ng` printed `Ign:` lines again and again and stayed at `0% [Working]`.
Cause	**gw01 was powered off.** gw01 is app01's only gateway to the internet (default route 192.168.60.1, NAT) – a **single point of failure**.
Fix	Powered on gw01; the install worked at once.
Lesson	When a server "cannot reach the internet", check the path hop by hop: `ip route` → ping the gateway → ping 8.8.8.8 → ping a name (DNS).

📸 **Evidence:**

```
Ign:1 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libipsec-mb1 amd64 1.2-1
Ign:2 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libjudydebian1 amd64 1.0.5-5
Ign:3 http://de.archive.ubuntu.com/ubuntu jammy/main amd64 libsctp1 amd64 1.0.19+dfsg-1build1
Ign:4 http://de.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 stress-ng amd64 0.13.12-2ubun
tu1
Ign:1 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libipsec-mb1 amd64 1.2-1
Ign:2 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libjudydebian1 amd64 1.0.5-5
Ign:3 http://de.archive.ubuntu.com/ubuntu jammy/main amd64 libsctp1 amd64 1.0.19+dfsg-1build1
Ign:4 http://de.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 stress-ng amd64 0.13.12-2ubun
tu1
Ign:1 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libipsec-mb1 amd64 1.2-1
Ign:2 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libjudydebian1 amd64 1.0.5-5
Ign:3 http://de.archive.ubuntu.com/ubuntu jammy/main amd64 libsctp1 amd64 1.0.19+dfsg-1build1
0% [Working]_
```

apt hangs

📸 **Evidence – 2 CPU cores; the runaway job started as job [1], PID 1918:**

```
bilal_002@app01:~$ sudo apt install -y htop stress-ng
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
htop is already the newest version (3.0.5-7build2).
stress-ng is already the newest version (0.13.12-2ubuntu1).
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcab-1.0-0 libgusb2 libmbim-glib4 libmbim-proxy libmm-glib0
  libqmi-glib5 libqmi-proxy libsmbios-c2 libtcl8.6 modemmanager tcl tcl8.6 usb-modeswitch
  usb-modeswitch-data
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 3 not upgraded.
bilal_002@app01:~$ nproc
2
bilal_002@app01:~$ yes > /dev/null &
[1] 1918
bilal_002@app01:~$ jobs
[1]+  Running                 yes > /dev/null &
```

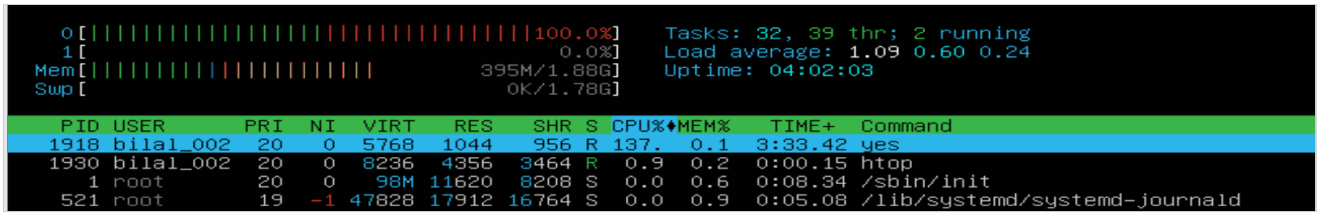
nproc yes

📸 **Evidence – Look: `yes` at the top with 99.9 % CPU, state R, owner `bilal_002` (shown cut as `bilal_0+`); without the minus in `--sort=-%cpu`, ps sorts lowest first:**

```
bilal_002@app01:~$ ps aux --sort=-%cpu | head -5
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
bilal_0+    1918 99.9  0.0   5768  1044 tty1     R    18:57   2:40 yes
root         711  0.5  0.4 314816  9312 ?        Ssl  14:59   1:15 /usr/bin/vmtoolsd
root        1557  0.3  0.0      0     0 ?        I    16:27   0:32 [kworker/0:0-events]
root        1550  0.2  0.0      0     0 ?        I    16:27   0:25 [kworker/1:1-events]
bilal_002@app01:~$
```

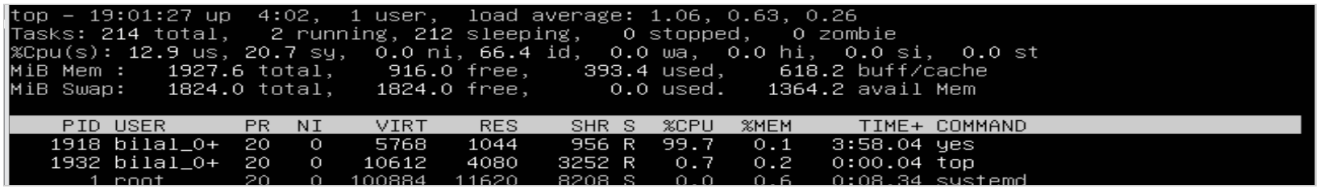
ps aux

📸 **Evidence – `htop`: core 0 at 100 %, core 1 at 0 % – `yes` fills exactly one of the two cores; load average 1.09:**



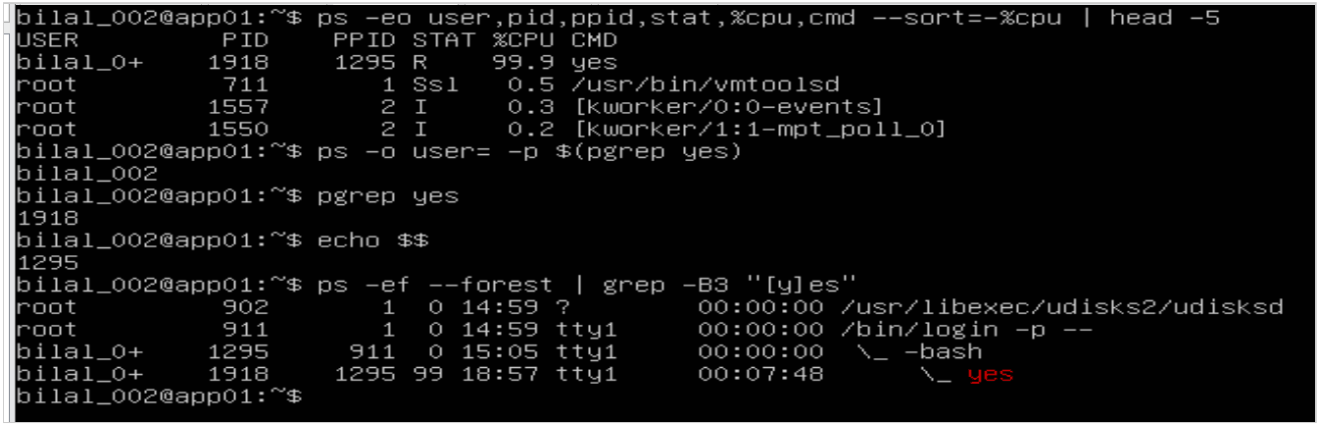
htop

📸 **Evidence – `top`: load average `1.06, 0.63, 0.26` – about 1.0 over the last minute and still rising over 5 and 15 minutes; `2 running, 0 zombie`:**



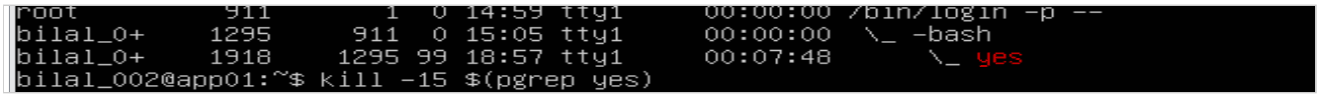
top

📸 **Evidence – Identify: PID 1918, PPID 1295; `echo $$` = 1295, so the parent is my bash; the family tree: `/bin/login` (console tty1) → `-bash` → `yes`. Over SSH, the parent of bash would be `sshd`:**



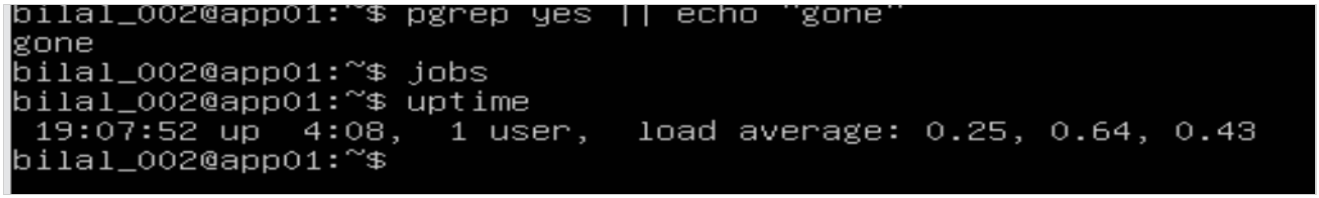
identify

📸 **Evidence – Stop politely with SIGTERM:**



kill 15

📸 **Evidence – Check: "gone", no jobs left, the load average falling (`0.25`, 0.64, 0.43 – the 1-minute value drops first):**



gone

Lesson: read the output **above** a success message. When `pgrep` was typed without a name it failed with an error – and `||` printed "gone" anyway. Only `pgrep yes || echo "gone"` is real proof.

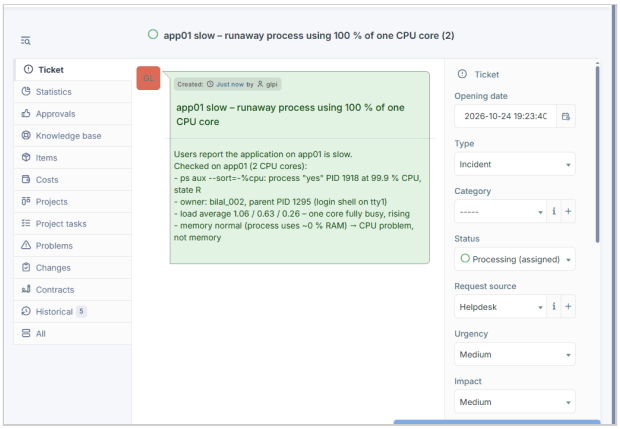
3.2 Record – GLPI ticket #2

📸 **Evidence – GLPI runs as an app + database pair on svc01: `glpi/glpi` publishes port 80 to the network; the database `mysql:8.0` shows only `3306/tcp` – not published, reachable only by GLPI inside Docker:**



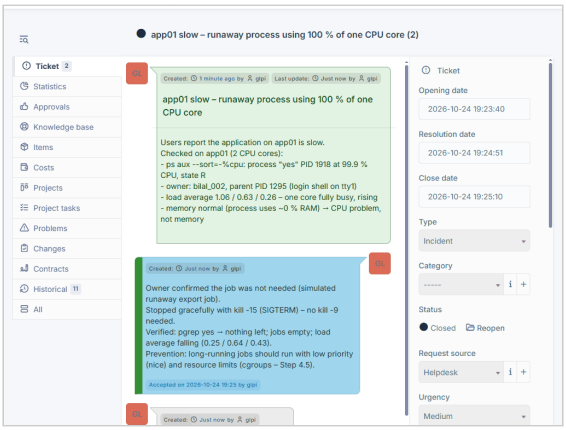
docker ps

📸 **Evidence – Incident ticket #2 created (Type Incident, Urgency/Impact Medium):**



ticket created

📸 **Evidence – solution added and approved, ticket closed (opened 19:23:40, solved 19:24:51, closed 19:25:10 UTC):**



ticket closed

Lesson: in a ticket, **Requester = who** reported it (a person), **Items = what** is broken (a computer from the inventory). In a company everyone works under a personal named account, not the built-in `glpi` super-admin.

Key understanding – the lab result (in my own words)

On app01 I first installed `htop` and `stress-ng`. The first attempt hung at `0% [Working]` with repeated `Ign:` lines because **gw01 was switched off**: gw01 is app01's only gateway to the internet (default route 192.168.60.1, NAT), so it is a single point of failure, and after powering it on the install worked. `nproc` showed **2 cores**. I created a runaway job with `yes > /dev/null &` (job [1], PID 1918). `ps aux --sort=-%cpu | head -5` put `yes` at the top with 99.9 % CPU, owner `bilal_002` (shown as `bilal_0+` because the name was cut off), state `R` (running), almost no memory (a CPU problem, not a memory problem) and a growing CPU TIME; without the minus in `--sort=-%cpu`, ps sorts lowest first and shows only idle processes. `htop` showed one core at 100 % and the other at 0 %, so `yes` fills exactly one of the two cores. `top` showed a load average of `1.06, 0.63, 0.26`: about 1.0 over the last minute (one core fully busy) and still rising over 5 and 15 minutes. `ps -eo user,pid,ppid,stat,%cpu,cmd` showed PID 1918 with PPID 1295, and `echo $$` printed 1295, so the parent of `yes` was my bash. `ps -ef --forest` showed the whole family: systemd (PID 1) → `/bin/login` (911, because I worked on the VMware console tty1; over SSH it would be `sshd`) → `-bash` (1295) → `yes` (1918). I stopped it gracefully with `kill -15` (SIGTERM): bash reported `[1]+ Terminated`, `pgrep yes || echo "gone"` printed "gone", `jobs` was empty, and `uptime` showed the load falling to `0.25, 0.64, 0.43`, the 1-minute value dropping first. I also learned to read the output above a success message: when I mistyped `pgrep` without a name, it failed with an error, and `||` printed "gone" anyway. Finally I recorded the incident in GLPI as ticket #2 – Type Incident, Requester = the person who reported it, Items = the computer that is broken – with the findings, the solution and a prevention note (low priority with `nice`, limits with cgroups), and closed it. `docker ps` on svc01 showed that GLPI is a real app + database pair: the `glpi/glpi` container publishes port 80, while its **MySQL 8.0** database shows only `3306/tcp`, not published, so users reach the app and only the app reaches the database.

Key understanding – the concept (in my own words)

1. The problem. The most common complaint about an application server is *"the application is slow"*. Very often the cause is one program using all of the server's CPU. To fix it, I must understand where the data lives, which job can cause the problem, and how to find and stop that job safely.

2. Where the data lives: files and records. Applications use two kinds of data. **Files** (uploads, photos, PDFs, CSV exports, backups) are kept on a **file server** and shared over the network with **NFS** (SMB on Windows); the application opens them as if they were on its own disk. **Records** (orders, customers, tickets)

are rows in tables inside a **database**, such as MariaDB or PostgreSQL. The application always talks to a database with **SQL**, both to write ("save this order") and to read ("show me today's orders"), and never with NFS. A server whose main job is running database software is called a **database server**. One database program can hold many separate databases (for example `shop`, `hr` and `glpi`), and two different database programs can even run on the same server, because each uses its own port (MariaDB 3306, PostgreSQL 5432). Most applications come as an **app + database pair**: the app contains the pages, buttons and **business logic** (the rules of what happens, for example "when an order is placed, reduce the stock"), and the database stores the data.

3. How a real company makes reports, and how it goes wrong. Users work in the application (a web shop, ERP or MES) on the **application server**. It writes and reads the live data in the **primary database**, always with SQL and only on the primary. The primary first writes every change into its **change log**, a numbered diary on disk (PostgreSQL: the **WAL**, write-ahead log; MariaDB: the **binlog**), and then changes the table. It streams that log **continuously over the network** to a **replica**, a read-only copy of the **same kind** of database (PostgreSQL → PostgreSQL, MariaDB → MariaDB), which **replays** the same changes on its copy, normally less than a second behind. This is **replication**. Every night an automatic **ETL job** (Extract, Transform, Load), started by a timer or cron, runs on a separate **job server**. It **extracts** millions of rows from the **replica**, **transforms** them (cleans and combines them) and **loads** them into a **data warehouse**, a separate database used only for reports. Moving data between different kinds of database like this is ETL, not replication. **Power BI** reads the data warehouse and shows dashboards to managers, such as sales, stock and KPIs. Typical jobs: *"export today's orders to a CSV file for accounting"*, *"create the monthly sales report"*, *"send new customers to another system every hour"*. This design keeps the heavy job away from the live system, so users feel nothing. If a company skips it, the heavy job slows the application in one of two ways. With a **direct effect**, the job runs **on the application server itself** and uses its CPU. With an **indirect effect**, the job reads the **primary database**, which becomes so busy that the application waits for its answers. A replica protects performance but is **not a backup**: if someone deletes data by mistake, the deletion is copied to the replica within seconds as well.

4. The Acme platform. Acme is smaller. **stor01 is a file and backup server, not a database server**: it holds `/data/app` (the customer uploads, on the LVM volume `lvdata`) and `/backups` (copies made by the nightly backup timer). app01 reads and writes these files through the NFS mount `/mnt/app`. The application and any export job **run on app01**, so a stuck job there is a **direct effect**: it uses app01's CPU, and the application becomes slow. Acme's real app + database pair is GLPI on **svc01**: every ticket, user and inventoried computer is a row in its **MySQL 8.0** database (a close relative of MariaDB), and GLPI reads and writes them with SQL. Only GLPI's port 80 is published to the network; the database port 3306 is reachable only by the GLPI container inside Docker, so users reach the app and only the app reaches the database.

5. Runaway job, hung job and the load average. A job causes trouble when it is **stuck in a loop**. A loop is a part of a program that repeats the same steps, such as "read the next row, write it, repeat", until it reaches its stop point, for example the last row. Because of a bug, the stop point never comes, so the job repeats forever at 100 % CPU, like a nightly export still running at 10 a.m. That is a **runaway job**: busy and out of control. A **hung job** is different: it uses almost no CPU because it is stuck **waiting**, like my Ctrl+Z waiting on NFS in Phase 3. A normal scheduled job, like my nightly backup, starts, finishes and stops. One **CPU core** runs one process at a time, and `nproc` shows how many cores a server has. The **load average** is the number of processes that want the CPU at the same moment, both running and waiting. `top` shows it as three numbers, the averages over the last **1, 5 and 15 minutes**. With 1 core, a load of 1.0 means fully busy, and 2.0 means one process is working while another waits. `3.0, 1.0, 0.2` means "busy only since a minute ago"; `3.0, 3.0, 3.0` means "busy for at least 15 minutes".

6. Processes, PID and PPID. Every running program is a **process** with its own number, the **PID**. The **PPID** is the PID of the process that **started** it, its parent. Processes form a family tree, shown by `ps -ef --forest`. **systemd** (PID 1) starts all services at boot, so services such as `sshd` have PPID 1 and no shell. When I log in over SSH, sshd starts a copy of itself for my session, and that copy starts my **shell**, bash, the program that reads the commands I type and runs them. Every command I start, for example `yes` or `systemctl restart sshd`, gets its **own PID**, and its **PPID is my bash**. Processes started by the same bash therefore share the same PPID.

7. Finding and stopping the job on app01, always in this order. Simulate → Look → Identify → Stop politely → Check → (kill -9 only as a last resort) → Record. Simulate: I first installed `htop` (a clearer live process viewer) and `stress-ng` (a test tool that creates CPU, memory or disk load on purpose, used later for limits). `yes > /dev/null &` creates a runaway job: `yes` prints the letter "y" without end, so it alone uses one full core; `>` sends this output to `/dev/null`, a special file where everything disappears and nothing is stored (it is not the `/bin` folder, which holds programs); and `&` runs it in the background. **Look:** `nproc` shows the number of cores, and `ps aux --sort=-%cpu | head -5`, `top` and `htop` list the processes using the most CPU. `yes` appears at the top with about 100 %, and the load average rises. **Identify:** `ps -eo user,pid,ppid,stat,%cpu,cmd` shows the **owner** (the user who started it), the PID, the PPID, the state (`R` = running), the CPU use and the **command** (`cmd`, the command line that started it, which shows what the program is). `ps -o user= -p $(pgrep yes)` shows only the owner, and `pgrep yes` shows only the PID. **Stop politely:** `kill -15` sends **SIGTERM**, a **graceful** stop: the program is asked to finish and clean up. **Check:** `pgrep yes || echo "gone"` prints "gone" when nothing is left (`||` means "if the first command finds nothing, run the second"). **Last resort:** `kill -9` sends **SIGKILL**, a **forceful** stop. The kernel removes the process at once and its clean-up never runs, so leftovers such as a lock file or a half-written file can remain; I use it only if a process ignores SIGTERM. **Record:** in a company, before stopping anything, I check who owns the

process and whether it is important, asking the owner if needed. Afterwards I write a GLPI ticket: what happened, who owned the process, what I did and the result.

4. Step 4.2 – pipes: output → input → output

💡 **The story:** a pipe (`|`) passes the output of one program straight into the next as its input – no file in between. Admins use it every day, and it saves real disk space when moving big data: compress and send in one go, instead of first writing a temporary archive.

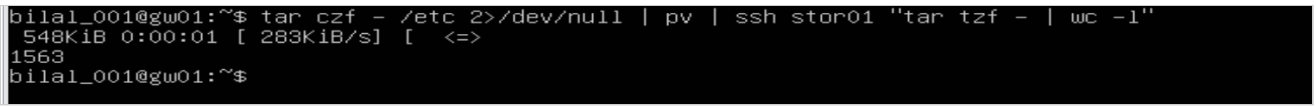
```
command 1 ──output──► | ──input──► command 2 ──output──► | ──input──► command 3 ──output──► screen
```

4.1 Hands-on – stream a compressed copy to another server (gw01 → stor01)

```
# gw01
sudo apt install -y pv
tar czf - /etc 2>/dev/null | pv | ssh stor01 "tar tzf - | wc -l"
```

Piece	Meaning
`tar czf - /etc`	**c**reate an archive, g**z**ip-compress it, **f**ile `-` = the pipe instead of a file
`2>/dev/null`	Hide the "permission denied" messages for root-only files
`pv`	**Pipe viewer** – shows the amount, speed (and % / ETA when the size is known)
`ssh stor01 "…"`	Carries the stream over the network and runs the command on stor01
`tar tzf - | wc -l`	On stor01: lis**t** what arrives and count the lines (`tar xzf - -C /data` would unpack = a real copy)

📸 **Evidence – 548 KiB streamed in 1 s; stor01 counted 1,563 files; nothing was stored on either server. `pv` shows `<=>` instead of % because a stream's size is not known in advance:**



tar pv ssh

Everyday pipe chains (`a | b | c`):

```
ps aux | sort -k3 -nr | head -3          # top 3 CPU users
grep bash /etc/passwd | wc -l            # how many users have a bash shell
pv shop.sql.gz | gunzip | mysql shop     # restore a dump with progress, no unpacked temp file
```

`|` (one bar) = pipe. `||` (two bars) = "or": run the second command only if the first fails – a different thing.

4.2 Reference only – named pipe (FIFO)

A FIFO is a **pipe with a name** – a tube between two programs **on the same server**. It **stores nothing** (size 0, type `p` in `ls -l`), data is gone once read (first in, first out), and the writer **waits** until a reader is there. It is needed only when two programs **cannot** be joined with `|` in one command line – a tool that accepts only a file name, or two programs started separately. Rare in daily admin work (examples: the Nagios/Icinga command file, some database export tools), so this part was kept as a reference and not run.

```
mkfifo /tmp/jobs                                                  # create the named pipe
ls -l /tmp/jobs                                                   # prw-r--r-- … 0 … /tmp/jobs
while read job < /tmp/jobs; do echo "processing: $job"; done      # terminal 1: the reader (worker)
echo "build-report" > /tmp/jobs                                   # terminal 2: the writer
rm /tmp/jobs                                                      # remove it

# the same transfer as above, as two separately started parts
mkfifo /tmp/stream
tar czf /tmp/stream /etc &                                        # writes "a file" — really the pipe
pv /tmp/stream | ssh stor01 "tar tzf - | wc -l"                   # reads it
rm /tmp/stream
```

Key understanding (in my own words)

A **pipe** (`|`) makes the output of one command the input of the next – **output → input → output** – with no file in between, like an assembly line, and only the last output reaches the screen. In `ps aux | sort -k3 -nr | head -3`, `ps` lists all processes, `sort` orders them by CPU and `head` keeps the top three. In real work pipes save time and disk space: on gw01, `tar czf - /etc | pv | ssh stor01 "tar tzf - | wc -l"` packed and compressed `/etc` into the pipe (`tar` packs many files into one stream: **c** create, **x** extract, **t** list, **z** gzip, **f -** = the pipe instead of a file), `pv` (**pipe viewer**) showed how much data passed and how fast (548 KiB in 1 second; a moving `<=>` instead of a percentage, because a stream's size is not known in advance), `ssh` carried the stream to stor01, and there `tar tzf -` listed what arrived and `wc -l` counted 1,563 files – nothing was stored on either server. With `tar xzf - -C /data` instead, stor01 would have

unpacked it, which is a real copy to a new server without any temporary archive; the pipe saves the temporary copy, but the target still needs space for what is finally stored there. `pv` is also used to show progress and time left for long jobs like restoring a database dump (`pv shop.sql.gz | gunzip | mysql shop`). One bar `|` is a pipe; two bars `||` mean "or" – run the second command only if the first fails. A **named pipe (FIFO)** is a pipe with a name in the filesystem: a tube between two programs on the same server that stores nothing (size 0, type `p` in `ls -l`), hands data over in order and only once, and makes the writer wait until a reader is there. It is needed only when two programs cannot be joined with `|` in one command line – for example a tool that accepts only a file name, or two programs started separately. It is rare in daily admin work, so I kept it as a reference: when I see a `p` at the start of an `ls -l` line, I know it is a pipe, not a normal file.

5. Next steps

Continued in Day 8.

1. VMware snapshot `40-automation` if not done yet.
2. Steps 4.3–4.5: signals and `trap` (why `kill -9` leaves a stale lock file), zombie processes, cgroups (`MemoryMax`, `CPUQuota`) – then the first incident drill.
3. Phase 2 tickets in GLPI.
4. At the end of the project: VMware ESXi / vCenter and Workstation's *Connect to Server*.

Project 2 – Acme Analytics Linux Platform

Day 8 · Phase 4: signals, lock, trap, a real queue system with a worker, L1 → L2 escalation

Muhammad Bilal Javed · 25 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	4 – Processes, signals and limits
Hosts worked on	stor01, svc01 (GLPI)
Status	Phase 4 🟡 – Steps 4.1, 4.2 ✅ (Day 7) · Step 4.3 🟡 – concept ✅, archive queue system Steps 1–7 ✅ (queue, producer, worker service, FIFO, request at any time, stop mid-job, L1 → L2 escalation with GLPI ticket #3); Step 8 (L3 Problem) and clean-up next
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary. Previous day: Day 7.

---

1. Step 4.3 – the concept: two copies, signals, lock and trap

💡 **In plain words:** jobs that run by themselves – nightly backups, exports – must never run **twice at the same time**, and when they are stopped they must not leave broken files behind. Three tools solve this: a **lock**, **signals** and **trap**.

Concept	Question it answers	Key line
Lock	Can a second copy start while the first is running?	`if [ -e "$LOCK" ]` · `flock -n` · a systemd service runs only once
Signal	How is a process stopped?	`kill -15` (SIGTERM) · Ctrl+C (SIGINT) · `kill -9` (SIGKILL)
trap	What is cleaned up when it is stopped?	`trap cleanup SIGINT SIGTERM`
`.part` file	How do I avoid a broken file that looks finished?	write to `file.part`, then `mv` when complete

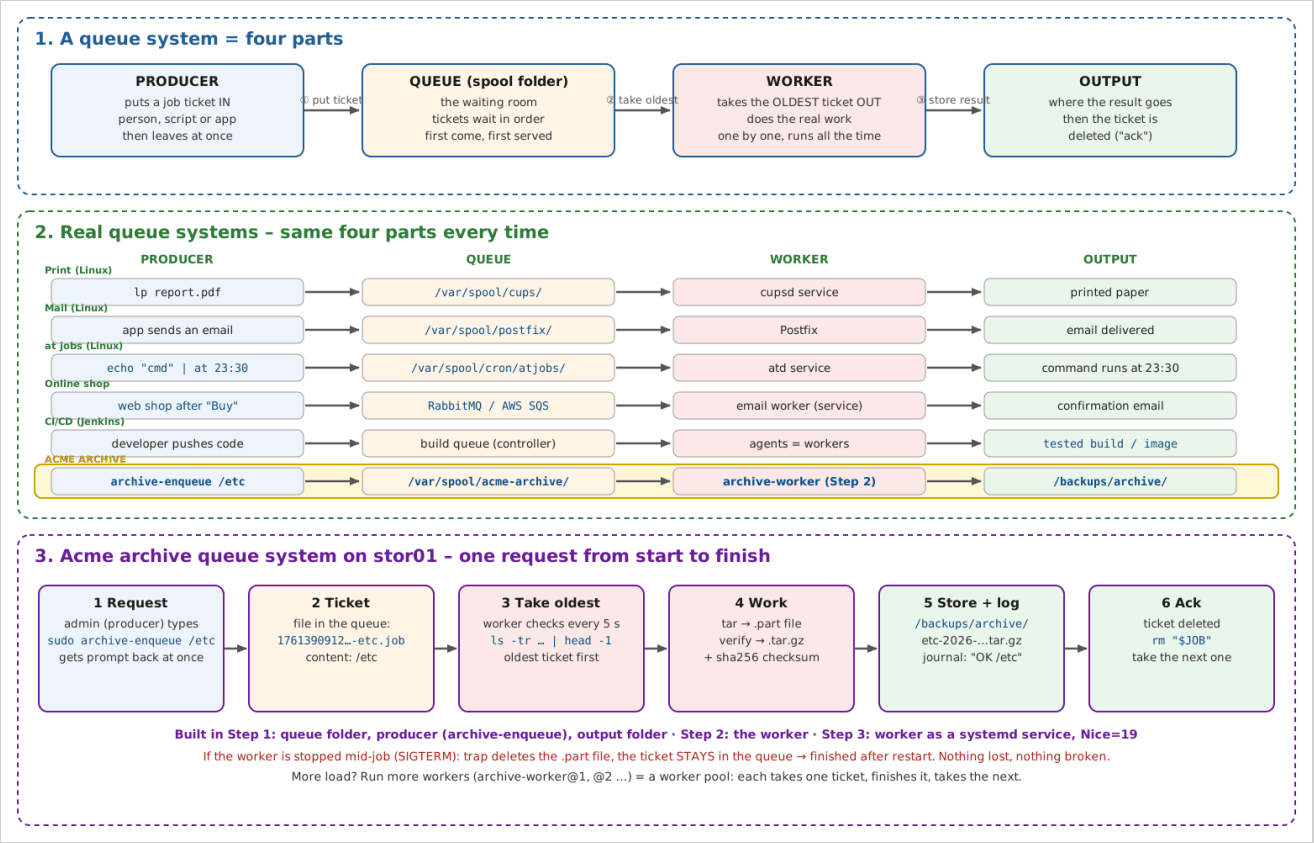
Key understanding (in my own words)

> A nightly backup or export started by a **timer or cron** must never run **twice at the same time**: if the timer's backup runs from 02:00 to 03:00 and I start it by hand at 02:05, two copies would write to the same place and produce a broken result. The protection is a **lock** – a "someone is working on this right now" sign. Some programs have a lock **built in** (apt and dpkg – which is why a second apt, often Ubuntu's automatic updates running in the background, answers *"Could not get lock"*), but simple tools like `cp` or `rsync` and my own scripts have **none**, so I add it myself: a lock file checked at the start, `flock -n` (a kernel lock released automatically even after `kill -9`), or simply a systemd service, which never runs twice. A lock only stops a **second copy from starting**; it never stops anyone from terminating the job, and once the first copy has finished and removed its lock, the job can run again normally. A job is stopped by a **signal** from the kernel: **SIGTERM** (`kill -15`, "please stop"; `systemctl stop` sends it), **SIGINT** (Ctrl+C) or **SIGKILL** (`kill -9`, "stop now"). Stopping affects resources in two ways: **CPU and memory** are freed by the kernel whichever signal is used, but files the job created on **disk** – its lock, temporary files, a half-written archive – stay behind unless clean-up code removes them. **Linux processes have no clean-up by default**: the default action of SIGTERM is "die immediately", and SIGTERM only **gives the program a chance** to clean up. rsync, apt and databases have that code built in, while `cp`, `scp` and a plain Bash script do not – stopping a 20 GB `cp` halfway leaves a broken half-file that looks complete. **`trap`** adds clean-up code to a script: `trap cleanup SIGINT SIGTERM` means "on SIGTERM or Ctrl+C, first run my `cleanup` steps, then exit", and it runs only the steps I write. Trap does not stop or restart the process – the signal stops it, the trap cleans the disk before it ends, so the next start begins clean. **SIGKILL cannot be trapped**, so after `kill -9` the leftovers stay; a leftover lock is a **stale lock**, and the fix is always: prove nothing is running (`pgrep -af <job>`), remove the lock by hand, start the job again. Good jobs also write into a temporary **`.part` file** and rename it only when it is complete and verified, so a half-written file never looks finished.

2. Step 4.3 – the concept: queue systems, producers and workers

💡 **In plain words:** a **lock** says "busy, go away". A **queue with a worker** says "wait in line, I'll do it for you": everyone drops a request and leaves at once, and one program works through the requests one by one, at low priority, without losing any.

📸 **Architecture – a queue system has four parts; Linux already has several (print, mail, `at`), and the same pattern runs online shops and Jenkins:**



queue systems

Concept	Question it answers
Scheduler (timer, cron)	**When** does a fixed job start? – driven by time
Socket	Start a service **live** for each connection – the caller waits for the answer
Queue	In **what order** do waiting jobs run? – FIFO, first in, first out
Producer	Who **puts** a job ticket in the queue? – a person, a script or an application
Worker	Who **does** the jobs, one by one, any time? – a program that runs all the time
nice / ionice	How much **CPU / disk** may it use? – a different question from the order

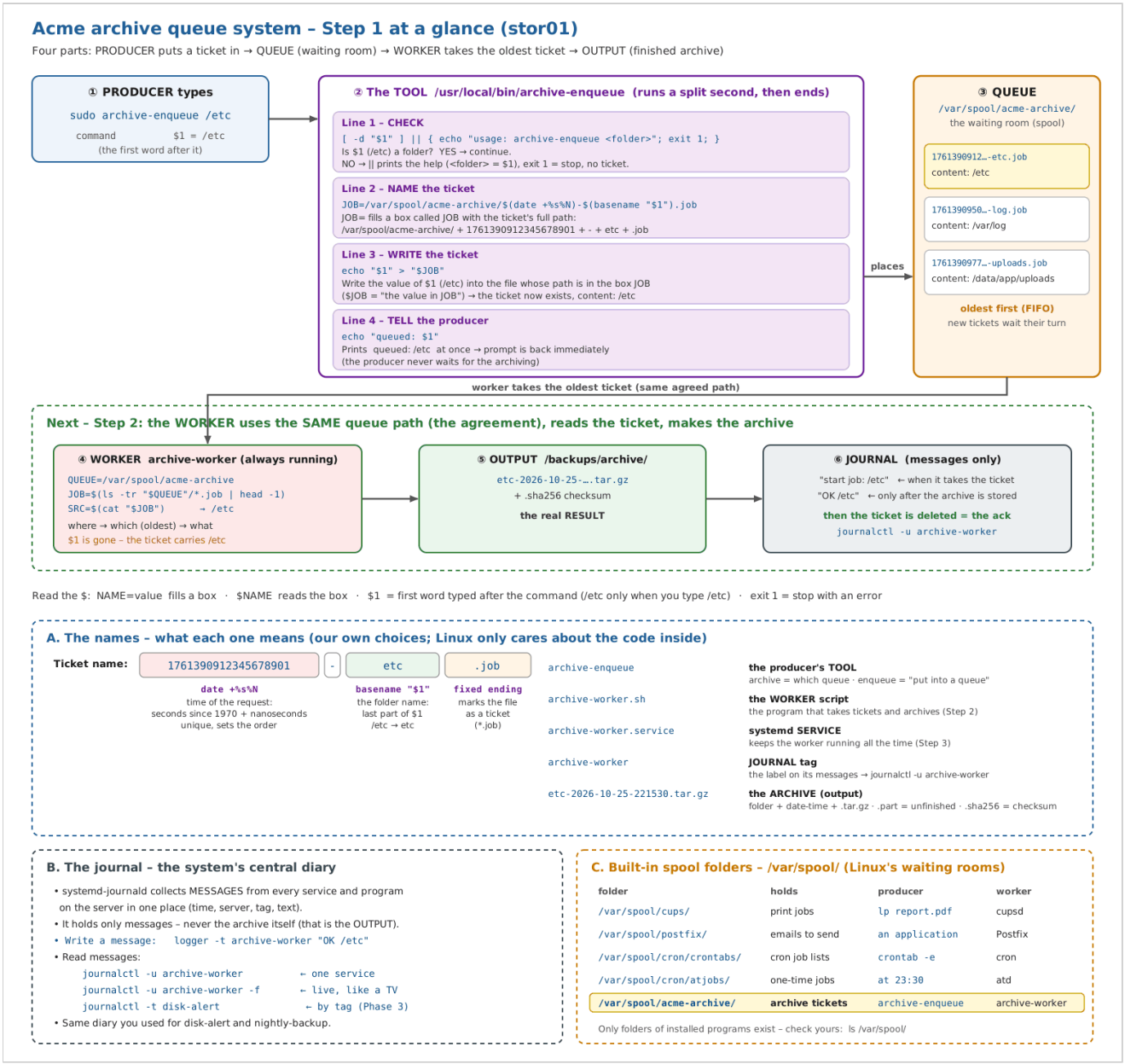
Key understanding (in my own words)

A **queue system** has four parts working together. A **producer** – a person, a script or an application – creates a **job ticket**, a small instruction such as *"archive /etc"*, places it in the **queue** and **gets control back at once**: it never waits for the job to be finished. The queue is a waiting room on disk called a **spool** folder; Linux keeps its own under **`/var/spool/`** – print jobs from `lp` wait in `/var/spool/cups/` for the worker `cupsd`, emails wait in `/var/spool/postfix/` for Postfix, `at` jobs wait in `/var/spool/cron/atjobs/` for `atd`. A **worker**, a program that runs all the time (usually a systemd service), takes the tickets **in order of arrival** – the oldest first, **FIFO**, first in, first out – does the work, stores the result in the agreed **output** place, and only then deletes the ticket (the **ack**, "done"), so if it fails or is stopped halfway the ticket stays and nothing is lost. Linux does not build a queue by itself: someone must place the tickets, and the worker must be written to take them out. Developers write workers; admins run and manage them – a systemd service with `Restart=on-failure`, more copies (a **worker pool**) when the load grows, logs in the journal, a safe stop with SIGTERM. Workers exist for three reasons: **nobody waits**, **the load stays under control**, **nothing is lost**. The same pattern runs a web shop (the shop puts *"send the confirmation email for order 501"* into RabbitMQ or AWS SQS and an email worker sends it) and **Jenkins** (the controller keeps the build queue, *"Waiting for next available executor"*, and agents are the workers). A **timer** is driven by **time** – one fixed job at a fixed time; a **socket** answers each **live connection** at once while the caller waits; a **worker** is driven by **stored requests** – any job, any time, one by one, while the requester has already left. Real automation grows step by step: do it by hand, script it, schedule it, and finally let a worker do it for everyone – repeating the same proven steps like a tireless user, without deciding anything new as an AI agent would. **`nice`** answers a different question from the queue: the queue decides the **order**, nice (-20 most important … 19 politest) decides **how much CPU** a running job gets.

3. Lab – the Acme archive queue system on stor01

💡 **The story:** at Acme, admins and scripts request archives (“archive /etc before I change the config”, “archive the uploads”). Instead of running heavy `tar` jobs by hand, they drop a **ticket** into a queue, and one **archive worker** turns the tickets into verified archives one by one, at low priority, so the customer application is never slowed down.

📸 **Architecture – one request from start to finish, the names, the journal and the built-in spool folders:**



step 1 overview

Part	Path
Producer’s tool	`/usr/local/bin/archive-enqueue`
Queue (spool folder)	`/var/spool/acme-archive/` – tickets like `1761390912345678901-etc.job`, content `/etc`
Worker	`/usr/local/bin/archive-worker.sh`, run as `archive-worker.service`
Output	`/backups/archive/` – `etc-2026-10-25-234228.tar.gz` + `.sha256`
Messages	the journal – `journalctl -u archive-worker`

3.1 Step 1 – the queue and the producer

```
# stor01
sudo mkdir -p /var/spool/acme-archive /backups/archive
sudo tee /usr/local/bin/archive-enqueue > /dev/null <<'EOF'
#!/bin/bash
[ -d "$1" ] || { echo "usage: archive-enqueue <folder>"; exit 1; }
JOB=/var/spool/acme-archive/$(date +%s%N)-$(basename "$1").job
echo "$1" > "$JOB"
echo "queued: $1"
EOF
sudo chmod +x /usr/local/bin/archive-enqueue
```

Line	Meaning
`[ -d "$1" ] || { echo "usage: …"; exit 1; }`	$1 = the first word typed after the command (the folder); `-d` asks "is it a folder?"; if not, `||` prints the help (`<folder>` describes $1) and `exit 1` stops – no ticket
`JOB=/var/spool/acme-archive/$(date +%s%N)-$(basename "$1").job`	JOB= fills a box with the ticket's path: queue folder + time of the request (seconds since 1970 + nanoseconds – unique, sets the order) + folder name (`basename` keeps the last part) + `.job`
`echo "$1" > "$JOB"`	Write the folder path into the ticket file ($JOB = the value in the box) – the instruction for the worker
`echo "queued: $1"`	Tell the producer at once – the prompt is back immediately

📸 **Evidence:**



step 1

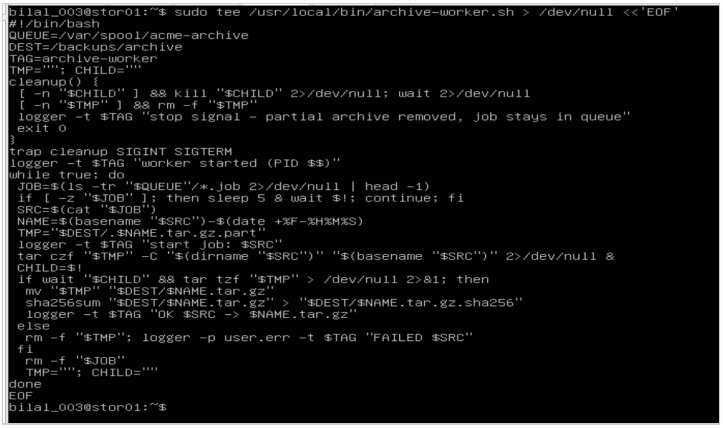
Lesson: my first version had `data` instead of `date`. `bash -n` did not catch it – it checks only the grammar of a script, not whether its commands exist. Only a test run proves a script works.

3.2 Step 2 – the worker

The full script is in the screenshot below. The lines that carry the concept:

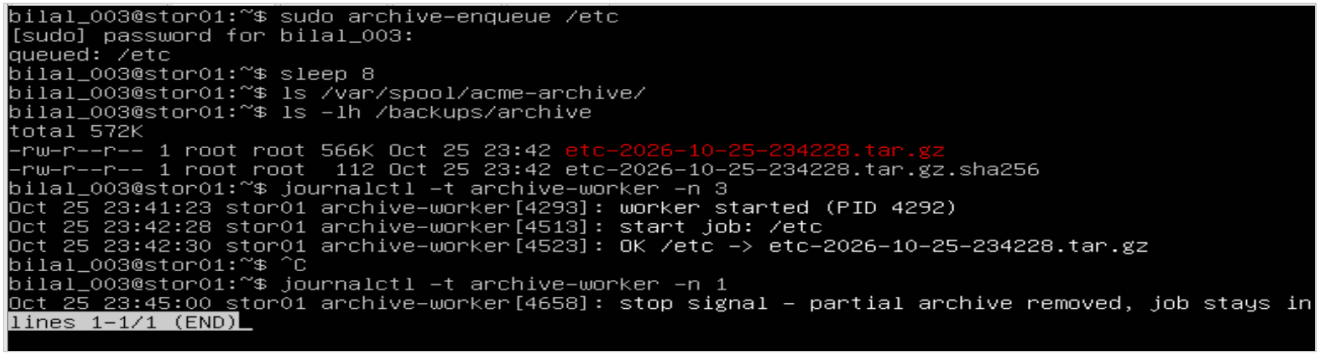
```
QUEUE=/var/spool/acme-archive                          # WHERE – the agreed queue folder (same path as the producer)
DEST=/backups/archive                                  # OUTPUT
trap cleanup SIGINT SIGTERM                            # TRAP – on SIGTERM/Ctrl+C: stop tar, delete the .part file
while true; do                                         # WORKER – runs forever
  JOB=$(ls -tr "$QUEUE"/*.job 2>/dev/null | head -1)   # WHICH – the oldest ticket first (FIFO)
  SRC=$(cat "$JOB")                                    # WHAT – read the folder from the ticket
  TMP="$DEST/.$NAME.tar.gz.part"                       # SAFE FILE – work into a .part file
  logger -t $TAG "start job: $SRC"                     # JOURNAL – message only
  mv "$TMP" "$DEST/$NAME.tar.gz"                       # OUTPUT – only a complete, verified archive gets the real name
  logger -t $TAG "OK $SRC -> $NAME.tar.gz"             # JOURNAL – "OK" only after the archive is stored
  rm -f "$JOB"                                         # ACK – delete the ticket
done
```

📸 **Evidence – the worker script:**



worker script

📸 **Evidence – manual test: worker running in one terminal; in the other `queued: /etc` at once, the queue empty again (ack), `etc-2026-10-25-234228.tar.gz` (566 K) + `.sha256` in the output, the journal *worker started* → *start job* → *OK* within 2 seconds; Ctrl+C → the trap's *"stop signal"* message:**



enqueue archive journal trap

Lessons: I first typed the worker by hand with five mistakes – `backups/archive` without the leading `/`, `[ -n "$CHILD"]` without the space before `]`, `SIGNIT`, `sha56sum` and, worst, **`*.JOB` instead of `*.job`**: Linux is case-sensitive, so the worker would have waited forever without any error. Long scripts are pasted, not retyped. The number in `archive-worker[4513]` is the PID of each `logger` call; the worker's own PID is the one in the message (4292).

3.3 Step 3 – the worker as a service, at low priority

```
sudo tee /etc/systemd/system/archive-worker.service > /dev/null <<'EOF'
[Unit]
Description=Acme archive worker (processes the archive queue)
After=local-fs.target
[Service]
ExecStart=/usr/local/bin/archive-worker.sh
Restart=on-failure
Nice=19
IOSchedulingClass=idle
[Install]
WantedBy=multi-user.target
EOF
sudo systemd-analyze verify /etc/systemd/system/archive-worker.service
sudo systemctl daemon-reload
sudo systemctl enable --now archive-worker
systemctl status archive-worker --no-pager
ps -o pid,ni,cmd -C archive-worker.sh
```

Setting	Meaning
`ExecStart=`	The worker program
`Restart=on-failure`	Restart it automatically if it crashes
`Nice=19` / `IOSchedulingClass=idle`	Lowest CPU priority / use the disk only when nobody else needs it (= `nice -n 19`, `ionice -c3`)
`WantedBy=multi-user.target` + `enable`	Start at every boot – a link in `multi-user.target.wants/`

📸 **Evidence – the unit; `verify` finds nothing in my file (the yellow line is Ubuntu's own `snapd.service`):**

```
bilal_003@stor01:~$ sudo tee /etc/systemd/system/archive-worker.service > /dev/null <<'EOF'
[Unit]
Description=Acme archive worker (processes the archive queue)
After=local-fs.target
[Service]
ExecStart=/usr/local/bin/archive-worker.sh
Restart=on-failure
Nice=19
IOSchedulingClass=idle
[Install]
WantedBy=multi-user.target
EOF
bilal_003@stor01:~$ sudo systemd-analyze verify /etc/systemd/system/archive-worker.service
/lib/systemd/system/snapd.service:23: Unknown key name 'RestartMode' in section 'Service', ignoring
bilal_003@stor01:~$ _
```

service unit

📸 **Evidence – `active (running)`, Main PID 4839, 1.6 MB; the cgroup holds the worker and its child `sleep 5` (empty queue, waiting); NI 19:**

```
bilal_003@stor01:~$ sudo systemctl daemon-reload
bilal_003@stor01:~$ sudo systemctl enable --now archive-worker
Created symlink /etc/systemd/system/multi-user.target.wants/archive-worker.service → /etc/systemd/sy
stem/archive-worker.service.
bilal_003@stor01:~$ systemctl status archive-worker
● archive-worker.service - Acme archive worker (processes the archive queue)
     Loaded: loaded (/etc/systemd/system/archive-worker.service; enabled; vendor preset: enabled)
     Active: active (running) since Sun 2026-10-25 23:53:46 UTC; 22s ago
   Main PID: 4839 (archive-worker.)
      Tasks: 2 (limit: 964)
     Memory: 1.6M
        CPU: 130ms
     CGroup: /system.slice/archive-worker.service
             ├─4839 /bin/bash /usr/local/bin/archive-worker.sh
             └─4860 sleep 5

Oct 25 23:53:46 stor01 systemd[1]: Started Acme archive worker (processes the archive queue).
Oct 25 23:53:46 stor01 archive-worker[4840]: worker started (PID 4839)
bilal_003@stor01:~$ ps -o pid,ni,cmd -C archive-worker.sh
    PID  NI CMD
   4839  19 /bin/bash /usr/local/bin/archive-worker.sh
bilal_003@stor01:~$
```

service running

Key understanding – the lab so far (in my own words)

On stor01 I built the **Acme archive queue system**. In **Step 1** I created the **queue** `/var/spool/acme-archive/` (the waiting room for job tickets), the **output** `/backups/archive/` and the **producer's tool** `archive-enqueue` in `/usr/local/bin`, the folder for an admin's own commands. When I type `sudo archive-enqueue /etc`, `$1` – a box filled with the first word after the command – becomes `/etc`; `[ -d "$1" ]` checks it is a folder (otherwise `||` prints the help and `exit 1` stops); `JOB=` builds the ticket's path from the queue folder, the time of the request (`date +%s%N`) and the folder name (`basename`); `echo "$1" > "$JOB"` writes the folder into that ticket file, and *"queued: /etc"* comes back at once. The tool's name is only a label for humans – "enqueue" is the standard word for "put into a queue" – and what it does is decided only by its code. In **Step 2** I created the **worker** `archive-worker.sh`. It knows where to look because the **same queue path** is written at its top – producer and worker simply agree on it, as `cupsd` reads its spool folder from its settings file – and it reads the folder from the ticket with `cat`, because `$1` exists only for the split second the producer runs. In a manual test it turned my ticket into `etc-2026-10-25-234228.tar.gz` with a `.sha256` checksum within 2 seconds, wrote *start job* and *OK* to the **journal** (the server's central diary, which holds only messages, never the archive itself), deleted the ticket (the ack), and on Ctrl+C its trap logged *"stop signal"* before exiting. In **Step 3** I made it a **systemd service** with `Restart=on-failure`, `Nice=19` and `IOSchedulingClass=idle`; it now runs all the time as PID 4839 at **NI 19**, its cgroup showing it waiting (`sleep 5`) for the next ticket. Because a service runs only once, systemd itself is the lock – no second worker can start.

3.4 Step 4 – three jobs in the queue

```
# Terminal 1 (stor01 console): journalctl -u archive-worker -f
# Terminal 2 (ssh stor01 from gw01):
df -h /backups
sudo archive-enqueue /data/app/uploads
sudo archive-enqueue /etc
sudo archive-enqueue /var/log
ls /var/spool/acme-archive/
journalctl -t archive-worker -n 10
```

📸 **Evidence – three tickets waiting, named by request time so the oldest comes first:**

```
bilal_003@stor01:~$ df -h /backups
Filesystem                    Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvbackups  2.9G  701M  2.1G  26% /backups
bilal_003@stor01:~$ sudo archive-enqueue /data/app/uploads
[sudo] password for bilal_003:
queued: /data/app/uploads
bilal_003@stor01:~$ sudo archive-enqueue /etc
queued: /etc
bilal_003@stor01:~$ sudo archive-enqueue /var/log
queued: /var/log
bilal_003@stor01:~$ ls /var/spool/acme-archive
1792974483564978473-uploads.job  1792974503083839724-etc.job  1792974511474086151-log.job
bilal_003@stor01:~$ ps -o pid,ni,cmd -C tar
    PID  NI CMD
bilal_003@stor01:~$
```

three tickets

📸 **Evidence – the live `-u` view shows *start job: /data/app/uploads* but not its *OK*:**

```
bilal_003@stor01:~$ journalctl -u archive-worker -f
Oct 25 23:53:46 stor01 systemd[1]: Started Acme archive worker (processes the archive queue).
Oct 25 23:53:46 stor01 archive-worker[4840]: worker started (PID 4839)
Oct 26 00:28:08 stor01 archive-worker[6564]: start job: /data/app/uploads
Oct 26 00:29:04 stor01 archive-worker[6594]: start job: /etc
Oct 26 00:29:05 stor01 archive-worker[6604]: OK /etc -> etc-2026-10-26-002904.tar.gz
Oct 26 00:29:05 stor01 archive-worker[6612]: start job: /var/log
```

journal -u

📸 **Evidence – `journalctl -t` (by tag) shows every message: uploads 00:28:08 → OK 00:29:04 (56 s), `/etc` 1 s, `/var/log` 11 s – strictly in arrival order (FIFO), one at a time:**

```
bilal_003@stor01:~$ journalctl -t archive-worker -n 10
Oct 25 23:42:28 stor01 archive-worker[4513]: start job: /etc
Oct 25 23:42:30 stor01 archive-worker[4523]: OK /etc -> etc-2026-10-25-234228.tar.gz
Oct 25 23:45:00 stor01 archive-worker[4658]: stop signal - partial archive removed, job stays in qu>
Oct 25 23:53:46 stor01 archive-worker[4840]: worker started (PID 4839)
Oct 26 00:28:08 stor01 archive-worker[6564]: start job: /data/app/uploads
Oct 26 00:29:04 stor01 archive-worker[6586]: OK /data/app/uploads -> uploads-2026-10-26-002808.tar.>
Oct 26 00:29:04 stor01 archive-worker[6594]: start job: /etc
Oct 26 00:29:05 stor01 archive-worker[6604]: OK /etc -> etc-2026-10-26-002904.tar.gz
Oct 26 00:29:05 stor01 archive-worker[6612]: start job: /var/log
Oct 26 00:29:16 stor01 archive-worker[6622]: OK /var/log -> log-2026-10-26-002905.tar.gz
lines 1-10/10 (END)
```

journal -t

Lesson: `logger` runs as a very short-lived process; sometimes it ends before the journal can link the message to the **service**, so `journalctl -u archive-worker` can miss a line. **`journalctl -t archive-worker`** filters by the **tag** set with `logger -t` and always shows them all.

3.5 Step 5 – a request at any time: worker vs timer

```
        sudo archive-enqueue /etc        # "archive /etc NOW – before I change the config"
        journalctl -t archive-worker -n 2
```

📸 **Evidence – queued, started and finished in the same second (00:34:12) – no waiting for a 02:00 timer:**

```
bilal_003@stor01:~$ sudo archive-enqueue /etc
queued: /etc
bilal_003@stor01:~$ sleep 3
bilal_003@stor01:~$ journalctl -t archive-worker -n 2
Oct 26 00:34:12 stor01 archive-worker[6922]: start job: /etc
Oct 26 00:34:12 stor01 archive-worker[6932]: OK /etc -> etc-2026-10-26-003412.tar.gz
bilal_003@stor01:~$ _
```

any time

3.6 Step 6 – stop the worker in the middle of a job (the trap test)

```
        sudo archive-enqueue /data/app/uploads
        sleep 5
        sudo systemctl stop archive-worker          # SIGTERM while tar is compressing 700 MB
        ls -la /backups/archive/                    # no .part file
        ls /var/spool/acme-archive/                 # the ticket is still there
        sudo systemctl start archive-worker
```

📸 **Evidence – after the stop: only complete archives in the output, no `.part` file:**

```
bilal_003@stor01:~$ sudo archive-enqueue /data/app/uploads
queued: /data/app/uploads
bilal_003@stor01:~$ sudo systemctl stop archive-worker
bilal_003@stor01:~$ ls -la /backups/archive
total 729364
drwxr-xr-x  2 root root      4096 Oct 26 00:36 .
drwxr-xr-x 12 root root      4096 Oct 25 10:51 ..
-rw-r--r--  1 root root    579024 Oct 25 23:42 etc-2026-10-25-234228.tar.gz
-rw-r--r--  1 root root       112 Oct 25 23:42 etc-2026-10-25-234228.tar.gz.sha256
-rw-r--r--  1 root root    579075 Oct 26 00:29 etc-2026-10-26-002904.tar.gz
-rw-r--r--  1 root root       112 Oct 26 00:29 etc-2026-10-26-002904.tar.gz.sha256
-rw-r--r--  1 root root    579075 Oct 26 00:34 etc-2026-10-26-003412.tar.gz
-rw-r--r--  1 root root       112 Oct 26 00:34 etc-2026-10-26-003412.tar.gz.sha256
-rw-r--r--  1 root root  10962843 Oct 26 00:29 log-2026-10-26-002905.tar.gz
-rw-r--r--  1 root root       112 Oct 26 00:29 log-2026-10-26-002905.tar.gz.sha256
-rw-r--r--  1 root root 734122617 Oct 26 00:28 uploads-2026-10-26-002808.tar.gz
-rw-r--r--  1 root root       116 Oct 26 00:29 uploads-2026-10-26-002808.tar.gz.sha256
bilal_003@stor01:~$ _
```

no part file

📸 **Evidence – the ticket stayed in the queue; the journal shows the trap's message:**

```
bilal_003@stor01:~$ ls /var/spool/acme-archive
17929749638379828 75-uploads.job
bilal_003@stor01:~$ journalctl -t archive-worker -n 3
Oct 26 00:34:12 stor01 archive-worker[6932]: OK /etc -> etc-2026-10-26-003412.tar.gz
Oct 26 00:36:08 stor01 archive-worker[7057]: start job: /data/app/uploads
Oct 26 00:36:43 stor01 archive-worker[7081]: stop signal - partial archive removed, job stays in qu>

bilal_003@stor01:~$ sudo systemctl start archive-worker
```

ticket stays

📸 **Evidence – the full story in the journal: start 00:36:08 → *Stopping…* → *stop signal – partial archive removed, job stays in queue* → *Deactivated successfully* (clean exit) → restart 00:38:24 (new PID 7094) → the same job → *OK* 00:39:32:**

```
bilal_003@stor01:~$ journalctl -u archive-worker -f
Oct 25 23:53:46 stor01 systemd[1]: Started Acme archive worker (processes the archive queue).
Oct 25 23:53:46 stor01 archive-worker[4840]: worker started (PID 4839)
Oct 26 00:28:08 stor01 archive-worker[6564]: start job: /data/app/uploads
Oct 26 00:29:04 stor01 archive-worker[6594]: start job: /etc
Oct 26 00:29:05 stor01 archive-worker[6604]: OK /etc -> etc-2026-10-26-002904.tar.gz
Oct 26 00:29:05 stor01 archive-worker[6612]: start job: /var/log
Oct 26 00:34:12 stor01 archive-worker[6922]: start job: /etc
Oct 26 00:34:12 stor01 archive-worker[6932]: OK /etc -> etc-2026-10-26-003412.tar.gz
Oct 26 00:36:08 stor01 archive-worker[7057]: start job: /data/app/uploads
Oct 26 00:36:43 stor01 systemd[1]: Stopping Acme archive worker (processes the archive queue)...
Oct 26 00:36:43 stor01 archive-worker[7081]: stop signal - partial archive removed, job stays in que
ue
Oct 26 00:36:43 stor01 systemd[1]: archive-worker.service: Deactivated successfully.
Oct 26 00:36:43 stor01 systemd[1]: Stopped Acme archive worker (processes the archive queue).
Oct 26 00:36:43 stor01 systemd[1]: archive-worker.service: Consumed 2min 1.131s CPU time.
Oct 26 00:38:24 stor01 systemd[1]: Started Acme archive worker (processes the archive queue).
Oct 26 00:38:24 stor01 archive-worker[7095]: worker started (PID 7094)
Oct 26 00:38:24 stor01 archive-worker[7102]: start job: /data/app/uploads
Oct 26 00:39:32 stor01 archive-worker[7125]: OK /data/app/uploads -> uploads-2026-10-26-003824.tar.g
z
```

stop restart ok

📸 **Evidence – the queue empty again after the restart:**

```
bilal_003@stor01:~$ ls /var/spool/acme-archive
17929749638379828 75-uploads.job
bilal_003@stor01:~$ journalctl -t archive-worker -n 3
Oct 26 00:34:12 stor01 archive-worker[6932]: OK /etc -> etc-2026-10-26-003412.tar.gz
Oct 26 00:36:08 stor01 archive-worker[7057]: start job: /data/app/uploads
Oct 26 00:36:43 stor01 archive-worker[7081]: stop signal - partial archive removed, job stays in qu>
bilal_003@stor01:~$ ls /var/spool/acme-archive
bilal_003@stor01:~$ ls -lh /var/spool/acme-archive
total 0
bilal_003@stor01:~$ _
```

queue empty

3.7 Step 7 – a deliberate escalation: L1 finds it, L2 fixes it

💡 **The story:** a colleague left manual database dumps in `/backups`, filling it. The archive worker fails. As **L1** I find the cause and collect evidence, but I am not allowed to delete someone else's data or resize volumes – so I **escalate**. Then, as **L2 – Linux Storage**, I get the owner's approval, fix it and close the ticket.

```
# create the problem
sudo fallocate -l $(df --output=avail -B1 /backups | tail -1) /backups/db-dump-manual.sql
sudo archive-enqueue /etc                          # still OK – see the lesson below
sudo fallocate -l $(( $(stat -f -c '%f' /backups) * $(stat -f -c '%S' /backups) )) /backups/db-dump-manual-2.sql
sudo archive-enqueue /var/log                      # now FAILED

# L1 – look, identify, do not fix outside the role
journalctl -t archive-worker -n 3
df -h /backups
sudo du -sh /backups/* | sort -h | tail -3
ls -l /backups/db-dump-manual*.sql
```

📸 **Evidence – L1: `FAILED` `/var/log`, `/backups` 100 %, the two dump files (owner root, 00:44 and 00:52):**

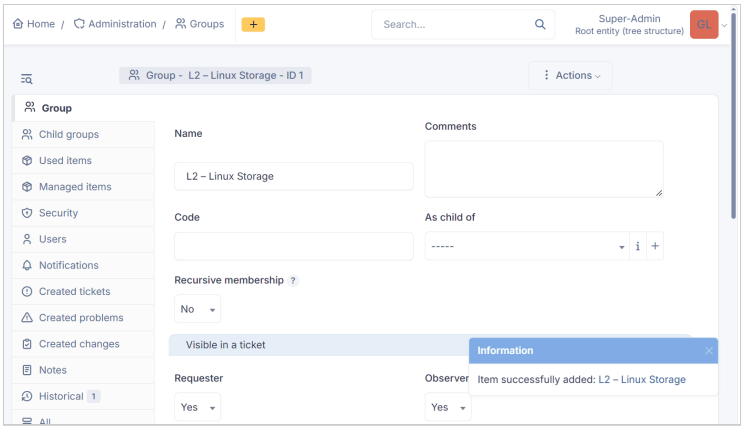
```
bilal_003@stor01:~$ sudo fallocate -l $(df --output=avail -B1 /backups | tail -1) /backups/db-dump-m
anual.sql
bilal_003@stor01:~$ sudo archive-enqueue /etc
queued: /etc
bilal_003@stor01:~$ sleep 5
bilal_003@stor01:~$ df -h /backups
Filesystem                  Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvbackups  2.9G  2.8G     0 100% /backups
bilal_003@stor01:~$ sudo fallocate -l $(($(stat -f -c '%f' /backups)* $(stat -f -c '%S' /backups)))
/backups/db-dump-manual-2.sql
fallocate: fallocate failed: No space left on device
bilal_003@stor01:~$ df -h /backups
Filesystem                  Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvbackups  2.9G  2.9G     0 100% /backups
bilal_003@stor01:~$ sudo archive-enqueue /var/log
queued: /var/log
bilal_003@stor01:~$ sleep 5
bilal_003@stor01:~$ journalctl -t archive-worker -n 3
Oct 26 00:44:38 stor01 archive-worker[7408]: OK /etc -> etc-2026-10-26-004437.tar.gz
Oct 26 00:53:25 stor01 archive-worker[7872]: start job: /var/log
Oct 26 00:53:25 stor01 archive-worker[7879]: FAILED /var/log
bilal_003@stor01:~$ sudo du -sh /backups/* | sort -h | tail -3
669M    /backups/db-dump-manual.sql
701M    /backups/2026-10-21
1.4G    /backups/archive
bilal_003@stor01:~$ ls -l /backups/db-dump-manual*.sql
-rw-r--r-- 1 root root 160464896 Oct 26 00:52 /backups/db-dump-manual-2.sql
-rw-r--r-- 1 root root 701198336 Oct 26 00:44 /backups/db-dump-manual.sql
bilal_003@stor01:~$
```

l1 evidence

Lesson – the 5 % root reserve: after the first dump `df` already showed **Avail 0 / 100 %**, yet the `/etc` archive still succeeded. ext4 keeps about **5 % reserved for root**; `df`'s "Avail" and "Use%" count only the space normal users can have, and the worker runs as root. Only after the second file filled **all** free blocks (`stat -f`) did the job fail. In a real incident this is why root services often keep working a little longer while users already see *"No space left on device"*.

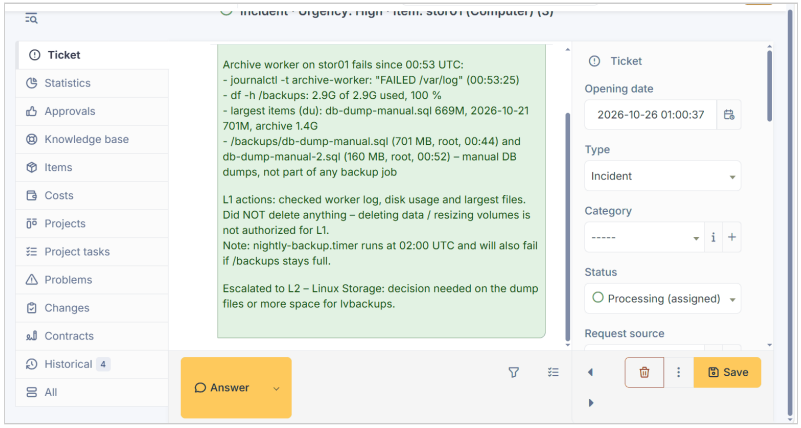
L1 in GLPI: a new group **L2 – Linux Storage**, and an **Incident** with Urgency **High**, Item **stor01**, the evidence, the L1 actions (*"did NOT delete anything – not authorized"*), a note that the 02:00 nightly backup would also fail, and **Assigned to: group L2 – Linux Storage**.

📸 **Evidence – the L2 group:**



group

📸 **Evidence – ticket #3 escalated: Incident · Urgency High · Item stor01 · Processing (assigned) to the L2 group:**

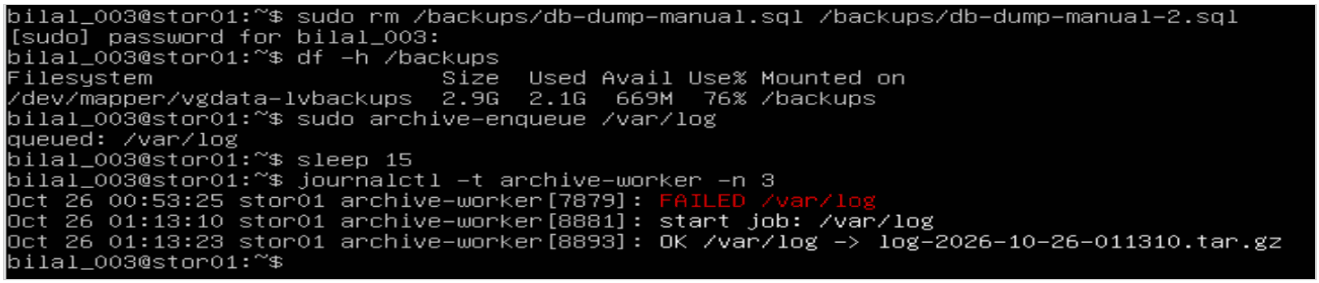


ticket 3

L2: a followup recording the owner's approval (*"both dumps are leftovers from a manual test – deletion approved"*), then the fix:

```
sudo rm /backups/db-dump-manual.sql /backups/db-dump-manual-2.sql
df -h /backups                          # 76 %, 669 MB free
sudo archive-enqueue /var/log           # re-queue the failed job
journalctl -t archive-worker -n 3
```

📸 **Evidence – space back, the failed job re-queued and `OK /var/log` at 01:13:23:**



l2 fix

Then the **solution** in ticket #3 – cause, decision, fix, verification, prevention – and the ticket was closed.

Role	Did	Did not
L1 – service desk	Found the cause (journal, `df`, `du`, `ls`), wrote the evidence, escalated to the right group	Delete data or change storage – not authorized
L2 – Linux Storage	Got the owner's approval in writing, removed the dumps, re-ran the job, verified, documented and closed	Fix the root cause in the design – that is L3's work (next)

Lesson – a lost job: the `/var/log` ticket disappeared after `FAILED`, because the worker deletes the ticket after a failure too. Real queue systems move failed jobs to a **failed queue** (a *dead-letter queue*) so they can be retried. That is a design improvement for **L3**.

Key understanding – Steps 4 to 7 (in my own words)

> With the worker running as a service, I queued three requests – `/data/app/uploads` (about 700 MB), `/etc` and `/var/log`. Each `archive-enqueue` answered *"queued"* at once, and the worker processed them strictly **in arrival order (FIFO)** and **one at a time**: uploads in 56 seconds, `/etc` the moment it finished, then `/var/log`; the later tickets simply waited. `journalctl -u archive-worker` missed one *OK* line, because a message from a very short-lived `logger` process cannot always be linked to the service, while `journalctl -t archive-worker` – filtering by the tag – showed every message. A request at any time (*"archive /etc now"*) was queued, started and finished within the same second, something a 02:00 timer cannot do. Then the real test: I stopped the worker with `systemctl stop` while it was compressing 700 MB. systemd sent SIGTERM, the worker's **trap** logged *"stop signal – partial archive removed, job stays in queue"*, the service was *Deactivated successfully*, `/backups/archive/` contained **no `.part` file**, and the uploads ticket was **still in the queue** – because the ticket is deleted only after success. After `systemctl start`, the new worker took that same ticket and finished it: nothing broken, nothing lost. Finally I practised a deliberate **escalation**. Two forgotten manual database dumps filled `/backups` – the first only up to the **5 % root reserve**, so `df` showed 100 % while the root-run worker could still write, and only the second made the job fail with *FAILED /var/log*. As **L1** I found the cause with `journalctl`, `df`, `du` and `ls`, did **not** delete someone else's data or resize the volume, and opened a High-urgency GLPI incident with all evidence, assigned to the group **L2 – Linux Storage**. As **L2** I recorded the owner's approval as a followup, removed the dumps (76 % used again), re-queued the failed job (*OK /var/log*), and wrote the solution – cause, decision, fix, verification, prevention – before closing the ticket. The failed `/var/log` ticket had been deleted by the worker, which showed a real design gap: production queues keep failed jobs in a **dead-letter queue** for retry. **L1's job is not to fix everything, but to find the cause, collect clear evidence, stay within its authority and hand over well; L2 fixes on the servers; L3 removes the root cause.**

4. Next steps

1. Step 4.3 Step 8: **L2 → L3** – a GLPI **Problem** for the root cause (dumps stored on the backup volume, no space monitoring on `/backups`, failed jobs deleted instead of kept), linked to incident #3, assigned to **L3 – Engineering**, status **Pending** with a workaround. Then Step 9: clean up `/backups/archive`.
2. Step 4.4 zombie processes (optional) · Step 4.5 cgroups (`MemoryMax`, `CPUQuota`) · the first incident drill · VMware snapshot `50-processes`.
3. Phase 2 tickets in GLPI.
4. At the end of the project: VMware ESXi / vCenter and Workstation's *Connect to Server*.

Linux processes, PIDs and zombies – from zero to the lab

Project 2 · Phase 4 · Step 4.4 · app01 (bilal_002) · read from top to bottom: each section builds on the one before

1. What is a process and a PID?

A **process** is a **program that is running right now**: my bash, `ls`, `tar`, sshd, the archive worker. Every process gets a **PID** (process ID), its **own number**, and a **PPID**, the **number of its parent** (the process that started it). **Every process has a parent**, all the way up to **systemd, PID 1**.

2. Where do PIDs come from? The pool

The **kernel** (the core of Linux) hands out PIDs like a **ticket machine at a counter**: it gives **the next number** to whatever starts, whether that's systemd, a kernel thread, a service, my bash or a command. There is **one pool for everything**, up to **`/proc/sys/kernel/pid_max`** (about 4 million on Ubuntu, 32768 on older systems).

> **Corrected (1 Nov):** numbers are given in **increasing order** (1823, 1824, 1825 ...). When a process ends, its number is **freed but NOT reused straight away** – the next process still gets the next **higher** number. Only when the counter reaches **pid_max** does it **wrap around** to low numbers, skipping the ones still in use. In `/proc/loadavg` (e.g. `0.01 0.07 0.03 1/238 1823`): load over 1/5/15 min · running/total **tasks incl. threads** · **last PID given**.

```
ticket machine: 1 (systemd) … 911 (login) … 1295 (bash) … 4808 … pid_max → wraps
cat /proc/sys/kernel/pid_max      # ★ size of the pool
cat /proc/loadavg                 # ★ last number = the PID given most recently
ps -e --no-headers | wc -l        # ★ how many PIDs are in use right now
```

3. How is a process born? fork · exec · wait

Word	Meaning	Picture
fork	the parent makes a **copy of itself**, a **new process with a new PID**	a cell divides into two
exec	"**replace me**": the process turns into another program, with the **same PID** and no new process	same person, new clothes
wait	the parent **reads the child's exit note** after it finishes, and the **PID is freed**	collecting the receipt

When I type `ls`, bash **forks** (copy, new PID), the copy **execs** into `ls`, `ls` finishes and leaves an exit note, and bash **waits** (reads it). That is why my prompt returns only after `ls` ends.

3b. Why fork + exec? bash works through a representative (clarified 1 Nov)

```
bash (1295) ── asks the KERNEL "copy me" (fork) ──► representative (new PID 4810)
     │                                                    │
     │                                  exec ──► becomes ls (still 4810)
     │                                                    │
     │                                finishes → leaves the exit note
     │
     └─ told (SIGCHLD) → READS the note (wait) → 4810 freed → prompt back
```

Why not exec directly? exec **replaces** the process. If bash exec'd into `ls` itself, bash would **become** `ls` and be gone when `ls` ends – my terminal would close. So bash **never does the work itself**: it asks the **kernel** to make a copy of it (with its environment, folder and open files) and a **new PID**, the copy turns into the command (keeping its PID), and bash stays alive. Safe proof: run `bash`, then `exec ls` – the inner bash disappears and `echo $$` shows the first bash again.

Who	Does what
kernel	makes the copy and gives the **new PID** (bash only asks)
representative (child)	execs into the command, does the work, ends by itself and **leaves** the exit note
bash (parent)	is told "your child finished" (SIGCHLD) and **reads** the exit note → PID freed. A **zombie** = the parent never reads it.

Built-ins are the exception: `cd`, `echo`, `exit` are done by **bash itself** – no representative, no new PID (that is why `cd` can change bash's own folder). Check with `type cd` → "shell builtin", `type ls` → `/usr/bin/ls`.

Not every program forks: only a process that **starts other programs** does fork + exec – bash (my commands), systemd (services), sshd (logins), my archive worker (→ `tar`). `ls` starts nothing, so it is the last link: `bash → ls`, `systemd → archive-worker.sh → tar`.

4. From boot to my shell on app01

```
kernel → systemd (1) → getty@tty1 —exec→ login (911) —fork→ bash (1295) → my commands
```

When app01 boots, the kernel starts **systemd** first, so it always has **PID 1** and is the **forefather of every process**. **tty1 is not a process**: it is the **terminal**, the console screen and keyboard in VMware (`tty` prints `/dev/tty1`). systemd starts the service **getty@tty1**, which shows "app01 login:". When I type my **username** (before the password), getty **execs into login**: the **same process (911)** changes its program, and nothing is passed to another process. After the correct password, login **forks** (a copy with a **new PID**), and the copy becomes **my bash (1295)**, with parent 911.

> **Clarified (1 Nov):** the kernel starts **only systemd**; **systemd** starts getty and all other services. **getty is a process** (a program that watches tty1 and shows "login:"); **tty1 is the terminal** itself (tty1, tty2 … = Terminal 1, 2 – **Ctrl+Alt+F2** opens tty2 with its own getty). After the password, login **does not disappear**: it **stays alive in the background**, sleeping, as bash's parent – that is why pstree shows `login(911)──bash(1295)`. I only *see* bash.

```
while I work:  ls ends   → leaves an exit note → bash reads it  → back to bash (prompt)
when I exit:   bash ends → leaves an exit note → login reads it → login ends
               → systemd starts a FRESH getty → new "login:" screen (new PIDs next time)
```

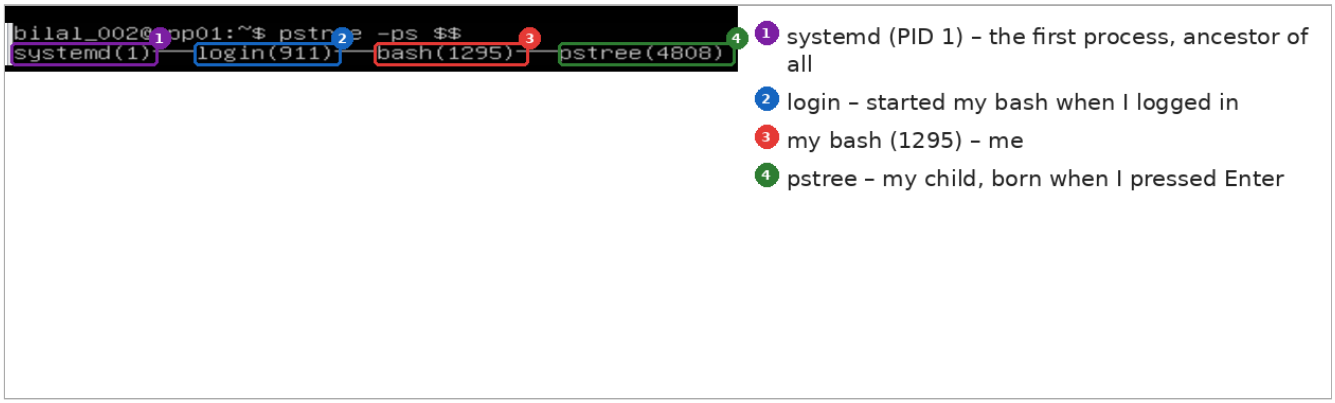
A living process has no exit note – the note is written only **at the moment a process ends**, and each level reports to the one above it. So `exit` closes the chain bash → login, and systemd puts a new login screen back. Test: `exit`, log in again, `echo $$` – a different number from 1295.

5. What I saw on app01 (my screenshots 1–7, with numbered marks)

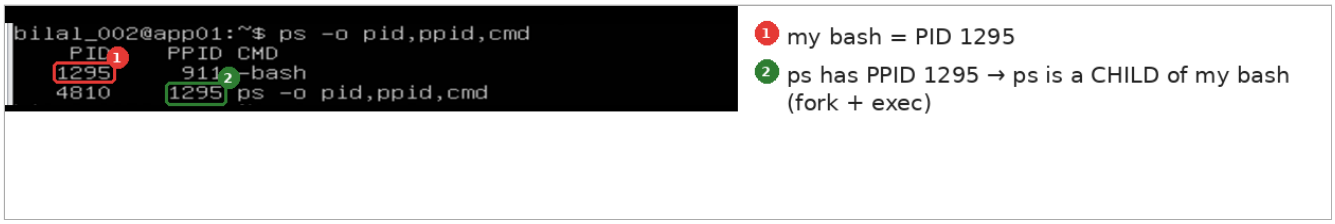
1 – Who am I: `echo $$` (**two dollars** = the PID of this shell) printed **1295**.



2 – Family tree: `pstree -ps $$` (**-p** = show PIDs, **-s** = show all parents) printed `systemd(1)──login(911)──bash(1295)──pstree(4808)`: **every command I run is a child of my shell**.



3 – Watch a birth: `ps -o pid,ppid,cmd` (**ps = process status**, **-o = output columns**) shows **only my terminal's processes**. `1295 911 -bash` = "**process 1295 is bash, and its parent is 911 (login)**", and the minus marks a **login shell**. `4810 1295 ps` = ps is the child of my bash (fork + exec).



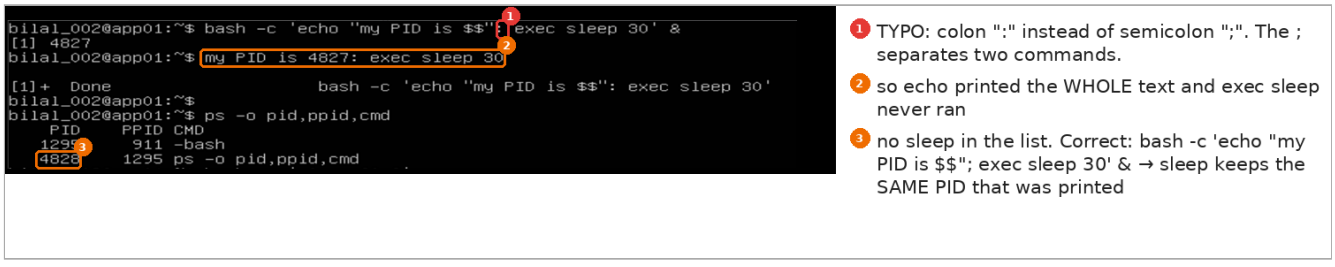
4 – Background child and a good parent: `sleep 30 &`: the **&** runs sleep (**4812**) in the **background**, so my prompt returned at once. I ran ps after more than 30 seconds, so sleep was already gone, the list only showed bash (`Ss`) and the ps command itself (**4813**, `R+`), and bash printed **`[1]+ Done`** (job 1, + = most recent): **no zombie, bash is a good parent**.



5 – Grandchild: `bash` started a **new bash (4814)** inside mine (parent 1295), and ps inside it (4820) had parent **4814**: **1295 → 4814 → 4820**. `exit` closed it.



6 – exec: `bash -c '…' &` (**small c = command**) starts a **new child bash** that runs the commands in the quotes and **ends by itself** (plain `bash` waits for me until `exit`). The **semicolon ;** separates two commands: `echo "my PID is $$"` prints **the new bash's PID** (echo is built into bash, so it makes no child), then **exec sleep 30** turns that bash into sleep **with the same PID**. My run failed only because I typed **":" instead of ";"**, so echo printed the whole text and exec never ran.



7 – Orphan: `bash -c 'sleep 60 &'`: the new bash started sleep and ended at once, so sleep lost its parent and **systemd adopted it**: `4830 1 S sleep 60` = **PPID 1, still alive**.



1. sleep 60, PID 4830 – its parent (inner bash) already exited
2. PPID = 1 → systemd ADOPTED it
3. STAT S = still alive (sleeping), not Z → ORPHAN, not zombie

STAT	**R** running (using CPU)	**S** sleeping (waiting, normal)	**T** paused (Ctrl+Z)	**Z** zombie	**+** foreground	**s** session leader

6. When is a PID freed, and when does it stay?

When a command (`tar`, `cp`, `ls`) is **completely done and its parent has read its exit note**, it has **no PID any more**: the number is **freed** back to the pool. Linux tells the parent "your child finished" (signal **SIGCHLD**), and bash **reads the note at once**. **`[1]+ Done` is only bash's report**, printed at the next prompt, so when I see it the process is **already gone**. The reader is always **the parent**: bash for my commands, the service itself for a service's helpers.

```
tar starts → gets PID 5100 → working (R/S) → finished → exit note waits
     ├── parent reads it (bash) ──────► PID FREED → back to the pool ✅
     └── parent never reads it (bug) ──► Z <defunct>, PID KEPT → zombie ❌
```

A PID stays in only two cases: the process is **still running** (R/S), or it is **done but its note was never read** (**Z**). Something that **looks** done but still has a PID is usually a command **paused with Ctrl+Z** (**T**). It continues with `fg` or ends with `kill %1`, while **Ctrl+C really ends it**.

7. Service PID vs command PID

The same kind of number from the same pool. Only **when, who and how long** are different:

	Services (sshd, cron, docker, archive-worker)	**Commands (ls, cp, tar)**
When taken	at **boot, first** → usually **small numbers**	later → **bigger numbers** (4808, 4812…)
Parent	**systemd, PPID 1**	**my bash**
How long	days, until stopped; **a restart gives a new PID**	seconds or minutes; a new PID every run
See it	`systemctl status ssh` → **Main PID**	`ps -o pid,ppid,cmd`

8. Normal, orphan, zombie

	Child	**Parent**	**What I see**	**Fix**
Normal	finished	read the exit note	`[1]+ Done`, gone	nothing
Orphan	**alive**, keeps working	died → **systemd adopts**	PPID **1**, STAT S/R	nothing
Zombie	**dead** (finished)	alive, **never read the note**	STAT **Z**, `<defunct>` (= "dead")	**restart the parent**; `kill -9` does nothing

Any process can become a zombie (tar, cp, ls, python, chrome), but **only after it has finished**, and **it depends on the parent, not the child**. A zombie uses **no CPU and no memory**, only **its PID and one slot**. The commands I type are **not** the problem, because **bash is a good parent** (in the labs `sleep` is only a **test tool**). Zombies appear in real life in three cases:

#	**Cause**	**Real example**
1	**Badly written program:** the developer forgot `wait()`	a Python/PHP/Node app starting a helper per request. My `zombie.py` copies this bug.
2	**Parent is stuck** (hung), so it can't read the notes	an app waiting forever on a dead NFS share or slow database
3	**Container without init**: no systemd to clean up	a Chrome scraper with thousands of `[chrome] <defunct>`. Fix: `docker run --init`

9. In the industry

Every zombie keeps its ticket, so thousands of them can **empty the PID pool**, and then the server shows **`fork: Resource temporarily unavailable`** and can't start anything new, not even an SSH login. Engineers notice it in monitoring or in the **“zombie”** count in `top`. They find the **parent through the PPID** (`ps -eo pid,ppid,stat,cmd | grep Z`) or `pstree`, **restart that service**, and open a **ticket for the developers** to fix the code. Docker start scripts end with **`exec python app.py`** so the app **replaces** the shell, becomes PID 1, and receives the stop signal (SIGTERM) directly.

Remember in one line each: PID = ticket from the kernel's pool · **fork** = new copy, **new PID** · **exec** = same process, new program, **same PID** · **wait** = parent reads the note, so the **PID is freed** · **zombie** = dead, note unread, **PID kept** · **orphan** = alive, parent gone, adopted by **PID 1** · **fix a zombie through its parent**.

Next: the zombie lab on app01 (Part 2 of the Step 4.4 PDF).

Zombie processes – what an admin must know

Project 2 · Phase 4 · Step 4.4 · one page instead of the full lab (rare in real life – know it, recognise it, fix it)

In my own words (corrected)

Every process leaves an exit note automatically when it ends – its **exit code** (0 = success; check the last one with `echo $?`). Nobody writes this note by hand: `ls`, `tar` or my own script all leave one. The **parent must read it** – in code that is the `wait()` call, which **reads** the note (it does not create it). If the parent **never reads** it – a bug (no `wait()`), a stuck parent, or a container without init – the finished child stays as a **zombie**: no CPU, no memory, only its **PID**. Zombies are rare, but thousands of them can use up all PIDs, and then the server cannot start anything new – not even an SSH login. `kill -9` on the zombie does nothing; the fix is to **restart the parent**, after which systemd adopts the zombie and reads its note.

Points to ponder – key clues

What is a zombie process?	A child that has **finished**, but whose parent has **not read its exit note**. It shows as STAT **Z** and `<defunct>`.
Does a zombie use CPU or memory?	**No.** It only keeps its **PID** and one slot in the process list.
Why is it a problem then?	Thousands of zombies can use up all PIDs → `fork: Resource temporarily unavailable` – no new process, not even an SSH login.
How do you find zombies?	`ps -eo pid,ppid,stat,cmd | grep -w Z` · or the **"zombie"** count in the `top` header.
Can you kill a zombie with kill -9?	**No** – it is already dead; the signal has nobody to receive it.
So how do you fix it?	Fix the **parent** (its PID is the zombie's **PPID**): restart that service. systemd (PID 1) adopts the zombie and reads the note at once. Long term: ticket to the developers – the code is missing `wait()`.
Zombie vs orphan?	**Zombie** = child dead, parent alive but never reads the note. **Orphan** = parent dead, child still **alive** → adopted by systemd (PPID 1), works normally.

What it looks like (example from the lab guide)

Finding it: STAT **Z**, `<defunct>`, and the PPID pointing to the parent (python3, PID 5012):

EXAMPLE – what app01 should show (your PID numbers will be different)

```
bilal_002@app01:~$ ps -eo pid,ppid,stat,cmd | grep -E "STAT| Z|defunct"
    PID    PPID STAT CMD
   5013    5012 Z    [python3] <defunct>
   5016    1295 S+   grep --color=auto -E STAT| Z|defunct
bilal_002@app01:~$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
   1295     911 Ss   -bash
   5012    1295 S    python3 /home/bilal_002/zombie.py
   5013    5012 Z    [python3] <defunct>
   5020    1295 R+   ps -o pid,ppid,stat,cmd
```

1. the zombie child, PID 5013
2. its parent is 5012 = python3 zombie.py
3. STAT Z = zombie
4. <defunct> = "dead". The process is finished, only its exit note waits
5. family view: bash 1295 → python3 5012 (alive, sleeping) → 5013 (Z, dead)

Fixing it: stop the parent → the zombie is gone, `top` shows "0 zombie":

EXAMPLE – what app01 should show (your PID numbers will be different)

```
bilal_002@app01:~$ kill 5012
[1]+  Terminated              python3 ~/zombie.py
bilal_002@app01:~$ ps -eo pid,ppid,stat,cmd | grep defunct
   5027    1295 S+   grep --color=auto defunct
bilal_002@app01:~$ top -bn1 | head -3 | grep Tasks
Tasks: 110 total,   1 running, 109 sleeping,   0 stopped,   0 zombie
bilal_002@app01:~$ rm ~/zombie.py
```

1. stop the PARENT (5012), the real fix
2. only the grep line itself is left → the zombie is GONE. systemd (PID 1) adopted it and read its exit note at once
3. top: "0 zombie"

cgroups – how a service gets its box and its limits

Project 2 · Phase 4 · Step 4.5 · example: archive-worker.service on stor01

① unit file (what I write)

```
[Service]
ExecStart=/usr/local/bin/archive-worker.sh

# priority – per PROCESS
Nice=19
IOSchedulingClass=idle

# limits – for the whole BOX
CPUQuota=30%
MemoryHigh=150M
MemoryMax=200M
TasksMax=20
```

reads

② systemd (PID 1)

starts the service
creates the box · writes the limits

creates + applies limits

③ the box (cgroup)

`/sys/fs/cgroup/system.slice/archive-worker.service/`

archive-worker.sh	tar czf …	gzip
PID 812 · nice 19	PID 4211 · nice 10 (renice)	PID 4212 · nice 19 (child)

one service = one box · all its processes and their children live here · Nice=19 is only the START value
each process can get its own nice (renice -n 10 -p 4211) · the limits are shared by the whole box

`CPUQuota 30%` over → THROTTLED slowed down: 30 ms of every 100 ms	`MemoryHigh 150M` over → slowed down soft limit · drop cache / swap · no kill
`MemoryMax 200M` over → OOM KILLER stops a process in THIS box only	`TasksMax 20` over → new fork FAILS "Resource temporarily unavailable"

④ other services = other boxes

`sshd.service · nightly-backup.service · glpi …`
never lose memory or CPU because of this box
→ one bad job cannot freeze the whole server

⑤ real example – compressing uploads at 14:00 while users work

1. **admin queues a big job**
   `sudo archive-enqueue /data/app/uploads`
2. **the worker starts tar + gzip**
   children inherit nice 19 and live in the SAME box
3. **gzip wants 100 % CPU**
   → throttled to 30 % · job ~3× slower, but finishes
4. **other services NOT slowed**
   sshd, GLPI, backup = own boxes · use the free 70 %
5. **gzip memory explodes > 200M**
   → OOM killer stops gzip only · job FAILED, trap cleans .part
6. **urgent? admin speeds tar up**
   `sudo renice -n 10 -p 4211 (box limit stays)`

Without the box: one compression job could slow the whole server for everyone. With it: the job is slower, everyone else is fine.

⑥ what MemoryHigh does – first drop CACHE, then use SWAP (same idea on Linux and Windows)

RAM before	process memory (really in use)	cache = files kept in RAM (original on disk)	free
RAM after step 1	process memory (really in use)	cache deleted from RAM → room again	
RAM after step 2	active memory	RAM → SWAP	idle program memory moved from RAM to swap – slow, nothing killed

Swap = a reserved area ON THE DISK (Ubuntu: /swap.img) used as extra slow RAM · check: free -h (RAM + swap) · swapon --show (where) · Windows: pagefile.sys

⑦ what "throttled" means – CPUQuota=30% (like mobile data after the limit: slower, not stopped)

30 ms run	70 ms paused	30 ms run	70 ms paused	30 ms run	70 ms paused	30 ms run	70 ms paused

every 100 ms the box may run only 30 ms → the job still finishes, about 3× slower · top shows ~30 %

⑧ the same cgroup limits in Docker and Kubernetes

	CPU ceiling (→ throttled)	memory ceiling (→ killed)	guaranteed minimum
systemd unit	`CPUQuota=30%`	`MemoryMax=200M`	– (`MemoryHigh` = soft limit)
docker run	`--cpus=0.3`	`--memory=200m`	–
docker-compose.yml	`cpus: 0.3`	`mem_limit: 200m`	–
Kubernetes YAML	`limits: cpu: "300m"`	`limits: memory: "200Mi"`	`requests: cpu / memory`

⑨ who creates the box – and the network – in Linux, Docker and Kubernetes

Linux service	`systemd` PID 1 – starts the service	→ `cgroup` one box per service			network: **the host's own network**
Docker	`dockerd` Docker engine	→ `containerd` manages containers	→ `runc` builds the container	→ `namespaces + cgroup` isolation + limits	network: **Docker networking**
Kubernetes	`kubelet` agent on each node	→ `containerd` manages containers	→ `runc` builds the container	→ `namespaces + cgroup` isolation + limits	network: **CNI plugin**

containerd = the manager (images, start/stop) · runc = the builder: namespaces = its own view of processes, network, hostname, files · cgroup = its limits
runc creates an EMPTY network namespace – the IP and connections come from Docker networking (bridge docker0, -p ports) or a Kubernetes CNI plugin (Calico, Flannel)

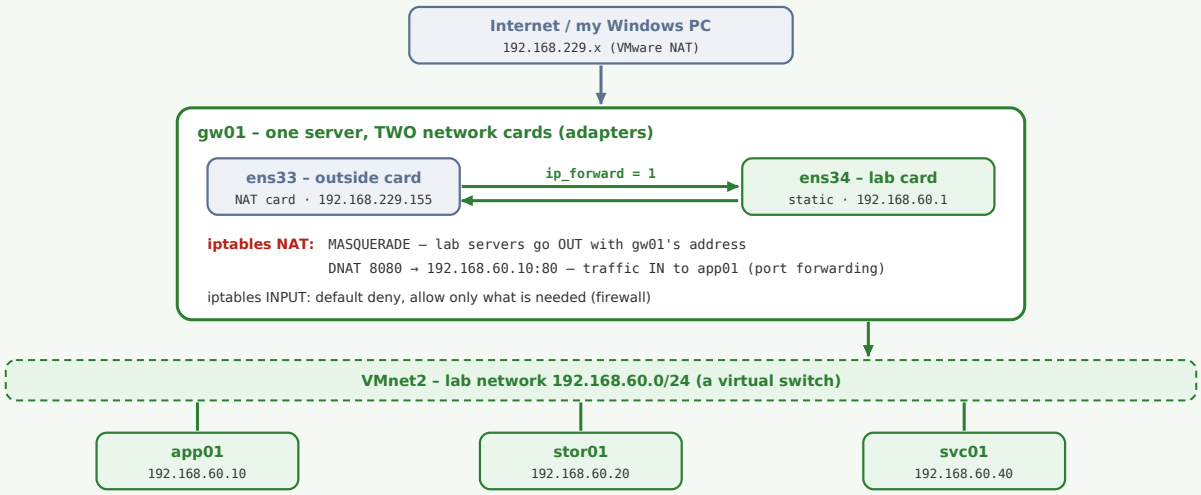
⑩ final paragraph (corrected) – nice, cgroups, memory, swap

nice sets the priority of each process (-20 highest, 19 lowest) – who goes first, with no ceiling. A cgroup is a box that systemd creates for every service: one service = one box, and all its processes and their children live in it. In the unit file, Nice=19 is only the start value for each process (renice can change one), while CPUQuota, MemoryHigh, MemoryMax and TasksMax are limits shared by the whole box. Over CPUQuota the box is throttled – it runs only e.g. 30 ms of every 100 ms, like mobile data after the limit: slower, never killed – and other services in their own boxes are not slowed. The CPU works only with RAM, so files read from disk are kept as copies in RAM, the cache. Above MemoryHigh (soft limit) the kernel first deletes this cache from RAM – nothing is lost, the originals are on the disk – and then moves idle program memory from RAM into swap, a reserved area on the disk (Ubuntu: /swap.img; Windows: pagefile.sys). Normal files on the disk are never touched; the box only gets slower. If it still reaches MemoryMax (hard wall), the kernel's OOM killer stops a process in this box only. TasksMax caps the number of processes and protects the PID pool. Docker (--memory, --cpus, mem_limit) and Kubernetes (requests = guaranteed, limits = hard ceiling, OOMKilled) use the same cgroups. Check RAM and swap with free -h, where swap lives with swapon --show, and the boxes live with systemd-cgtop. Who creates the box: on Linux systemd; in Docker dockerd → containerd → runc; in Kubernetes kubelet → containerd → runc. containerd manages, runc builds the container with namespaces (isolation) and a cgroup (limits); the IP comes from Docker networking or a CNI plugin.

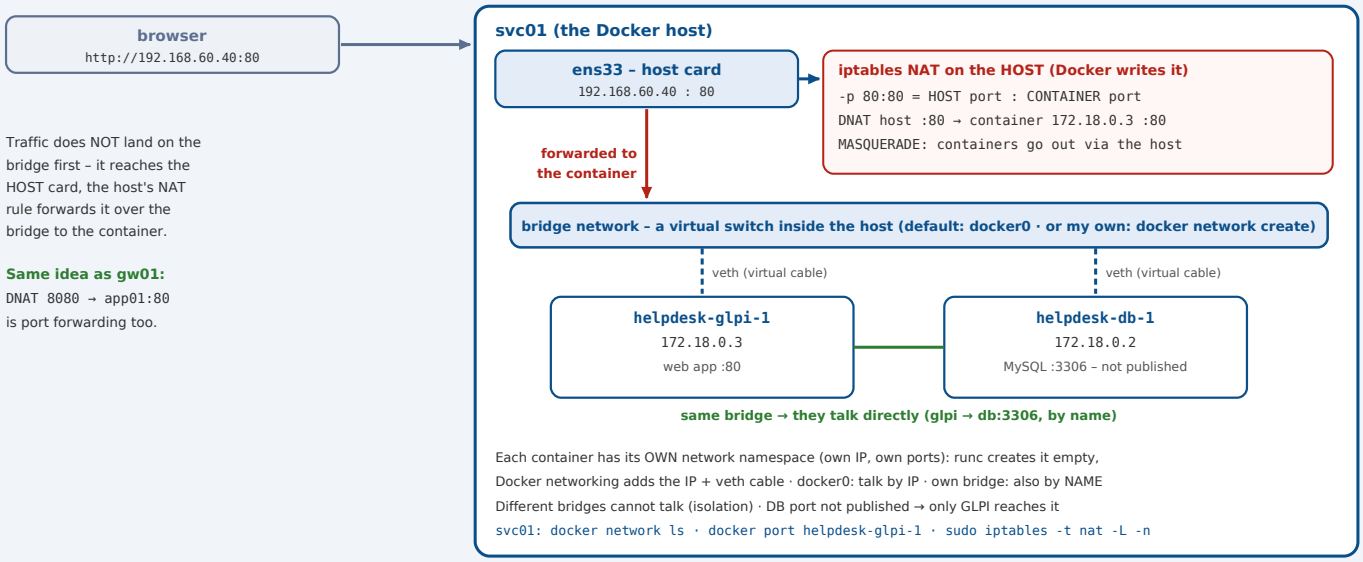
Networking – how traffic reaches a server, a container and a pod

Project 2 · Phase 4 · Step 4.5 (page 2) · the same idea three times: a network card / bridge + NAT rules that forward traffic

A · Linux server as router – my gw01 (Phase 1)



B · Docker – my GLPI on svc01: a bridge inside the server

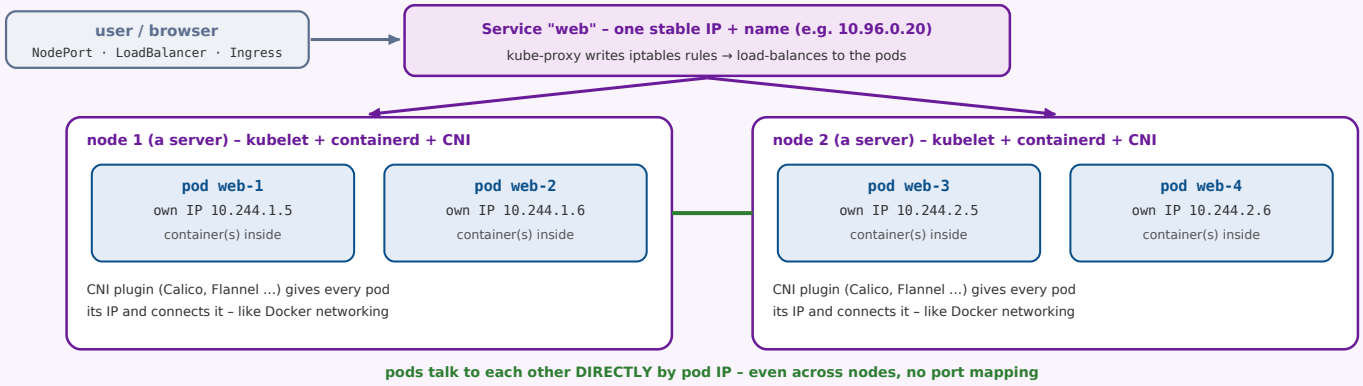


Traffic does NOT land on the bridge first – it reaches the HOST card, the host's NAT rule forwards it over the bridge to the container.

Same idea as gw01:
`DNAT 8080 → app01:80` is port forwarding too.

Each container has its OWN network namespace (own IP, own ports): runc creates it empty, Docker networking adds the IP + veth cable · docker0: talk by IP · own bridge: also by NAME
Different bridges cannot talk (isolation) · DB port not published → only GLPI reaches it
`svc01: docker network ls · docker port helpdesk-glpi-1 · sudo iptables -t nat -L -n`

C · Kubernetes – many servers (nodes), one flat pod network



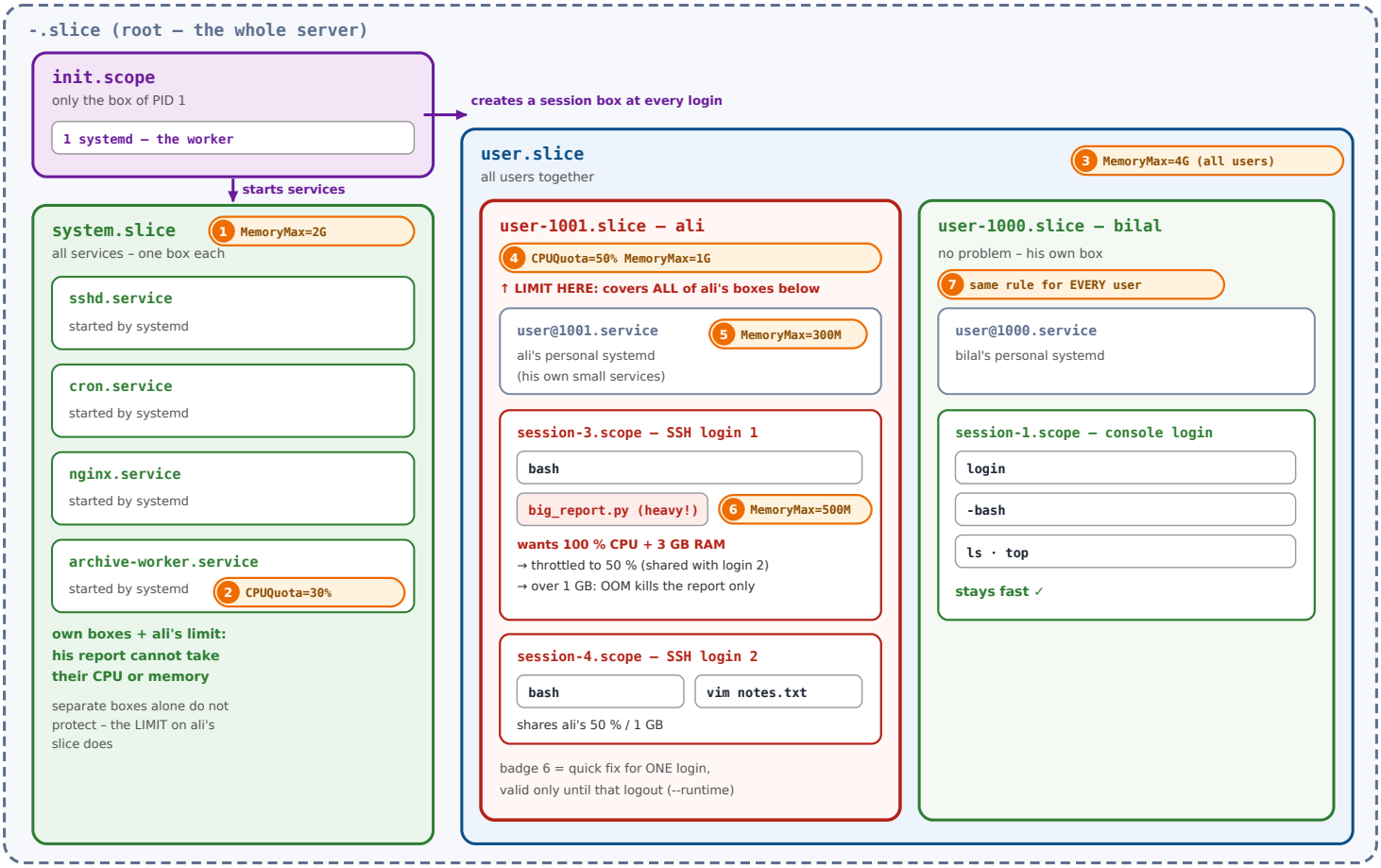
Pods come and go (new IPs), so clients use the Service name instead. Outside access: NodePort (a port on every node), LoadBalancer (cloud), or Ingress (HTTP routing by name).
`Check: kubectl get pods -o wide (pod IPs) · kubectl get svc (service IPs) · kubectl get ingress`

The same three ideas everywhere

	Linux (gw01)	Docker	Kubernetes
"switch" connecting machines	VMnet2 lab network	bridge (docker0 / my own)	CNI pod network
who gives the IP	static IP in netplan	Docker networking	CNI plugin
traffic IN from outside	DNAT 8080 → app01:80	-p 80:80 (DNAT on host)	Service + NodePort / Ingress
traffic OUT to the internet	MASQUERADE	MASQUERADE	node NAT (via CNI)

Two users on one shared server – where the boxes are and where to set limits

ali (UID 1001) has 2 SSH logins and runs a huge report · bilal (UID 1000) has 1 login · orange badges 1–7 = every place a limit can go



Where to set the limits – the 7 cases

1	**all services together**	`sudo systemctl set-property system.slice MemoryMax=2G`	permanent
2	**one service**	`sudo systemctl set-property archive-worker.service CPUQuota=30%`	permanent
3	**all users together**	`sudo systemctl set-property user.slice MemoryMax=4G`	permanent
4	**one user (ali)**	`sudo systemctl set-property user-1001.slice CPUQuota=50% MemoryMax=1G`	permanent
5	**one user's own services**	`sudo systemctl set-property user@1001.service MemoryMax=300M`	permanent
6	**one login only**	`sudo systemctl set-property --runtime session-3.scope MemoryMax=500M`	until logout
7	**every user, each separately**	`file /etc/systemd/system/user-.slice.d/limits.conf → [Slice] MemoryMax=1G`	permanent

Final paragraph

The whole server is one tree of boxes under -.slice. Directly under the root, side by side, are init.scope (only PID 1 – systemd, the worker), system.slice (all services, one box each) and user.slice (all logged-in users). systemd lives in init.scope but puts what it starts elsewhere: every service into system.slice, and at every login a new session box into the user's slice. On the user side the order goes from big to small: user.slice → user-1001.slice (ali) → user@1001.service (his own services) and session-3.scope / session-4.scope (his logins); the processes live only in the innermost boxes, and a child starts in its parent's box. A limit can be put on any box and covers everything inside it: all services together (1) or one service (2); all users together (3), one user (4), that user's own services (5), one login until logout (6), or the same rule for every user through a template (7). A limit on an outer box is ONE shared ceiling – the boxes inside compete for it; for a fixed share, give a box its own limit. Separate boxes alone do not protect: the limits do, and only root can set them. So when ali's report wants 100 % CPU and 3 GB, the limit on his slice throttles it to 50 % and the OOM killer stops it above 1 GB – bilal and every service keep running normally.

Where the limits are stored – set once, applied automatically

page 2 · limits are FILES on the disk · systemd reads them at boot and every time a box is created (like cloud-init applying its config automatically)

Do I have to set a limit again at every login? – No. At every login systemd creates a new **session box**, but it creates it **inside the user's slice** (e.g. `user-1001.slice`). The limit is stored on that slice in a file, so the new session is **automatically** inside the limit – nothing to do per login. Only badge 6 (a limit on one session itself) disappears at logout – it is an emergency fix, not the normal way.

How a login gets its limits automatically

```
ali logs in (SSH)
   → systemd creates session-5.scope  INSIDE  user-1001.slice
   → user-1001.slice already has its limit from a file:  MemoryMax=1G
   → session-5 and everything ali starts in it is inside the 1G ceiling – automatically
```

The files – one place per level

#	level	file on the disk	content
1	all services together	`/etc/systemd/system/system.slice.d/limits.conf`	`[Slice]` `MemoryMax=2G`
2	one service	`/etc/systemd/system/archive-worker.service.d/limits.conf` *or the unit file itself*	`[Service]` `CPUQuota=30%`
3	all users together	`/etc/systemd/system/user.slice.d/limits.conf`	`[Slice]` `MemoryMax=4G`

4	one user (ali)	`/etc/systemd/system/user-1001.slice.d/limits.conf`	`[Slice]`
			`CPUQuota=50%`
			`MemoryMax=1G`

5	one user's own services	`/etc/systemd/system/user@1001.service.d/limits.conf`	`[Service]` `MemoryMax=300M`
6	one login only	`/run/systemd/system.control/session-3.scope.d/…` (in RAM)	gone at logout / reboot
7	every user, each separately	`/etc/systemd/system/user-.slice.d/limits.conf` ← the dash = a **template** for every `user-*.slice`	`[Slice]` `MemoryMax=1G`

Two ways to write them

```
# A) let systemd write the file for you (permanent):
sudo systemctl set-property user-1001.slice MemoryMax=1G
#    → saved in /etc/systemd/system.control/user-1001.slice.d/

# B) write the file yourself, then reload:
sudo mkdir -p /etc/systemd/system/user-.slice.d
sudo nano /etc/systemd/system/user-.slice.d/limits.conf
sudo systemctl daemon-reload
```

Check what is active

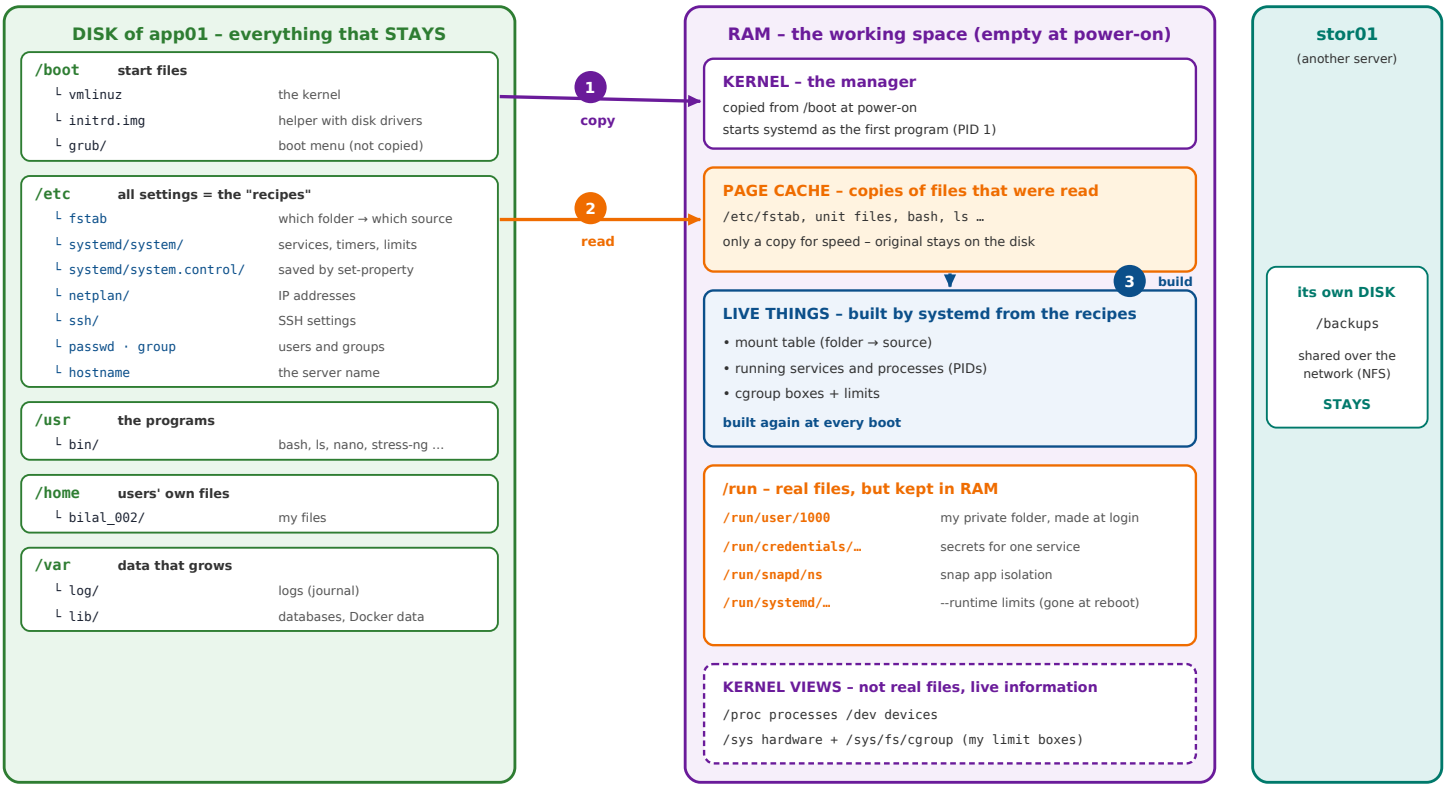
```
systemctl cat user-1001.slice            # all files for this box
systemctl show user-1001.slice -p MemoryMax -p CPUQuotaPerSecUSec
systemd-cgtop                                 # live usage per box
cat /sys/fs/cgroup/user.slice/user-1001.slice/memory.max
```

.d folder = a “drop-in” folder: extra settings added on top of a unit without changing the original file – the same as for your services in Phase 3.

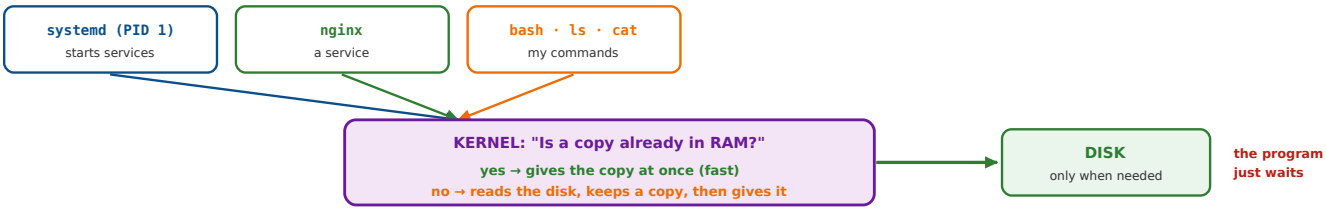
In one line: limits live in small files under `/etc/systemd/system/<box>.d/`; systemd applies them at boot and to every new box it creates – so one file for `user-.slice` limits every user, every login, forever. Only `--runtime` limits live in `/run` and disappear.

3 Nov · page 1 of 2 – app01 from power-on: disk, RAM, kernel, systemd and mounts

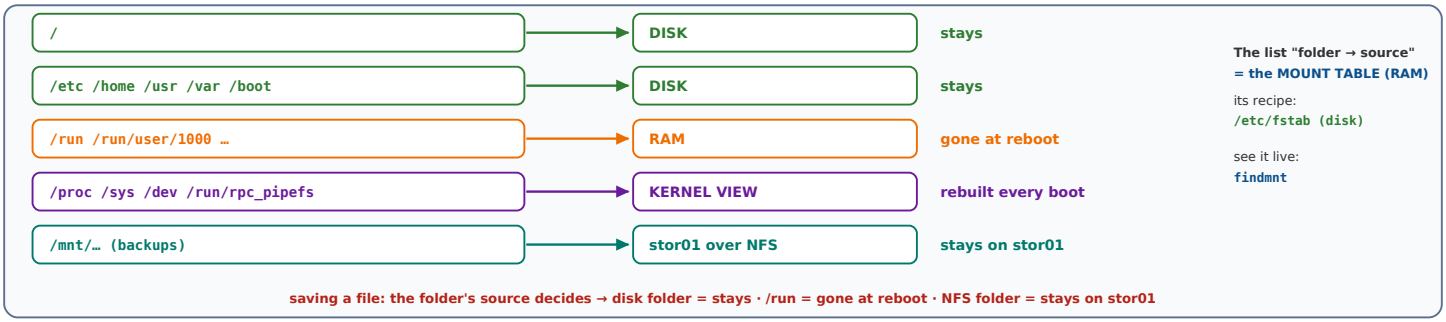
green = on the disk (stays) · purple/orange = in RAM (gone at power-off) · teal = on another server · ① ② ③ = the order at boot



Who asks the kernel for a file? – every program, directly



One folder tree – every folder is connected (mounted) to a source, and the SOURCE decides if it stays



3 Nov · page 2 of 2 – live test vs permanent, and my memory-limit test on app01

the big idea of the day: the work always happens in RAM – WHERE you write it decides if it stays

A · Every setting has two ways: LIVE (a test, gone at reboot) or a FILE in /etc (stays)

the change	LIVE – works at once · gone at reboot	FILE on the disk – read at every boot · stays
IP forwarding (gw01)	`sysctl -w net.ipv4.ip_forward=1 (/proc/sys/…)`	`/etc/sysctl.conf`
a service	`systemd-run --unit=mytest …`	`/etc/systemd/system/mytest.service + enable`
a limit	`systemd-run -p MemoryMax=… · set-property --runtime`	`set-property · …service.d/limits.conf`
a mount	`mount 192.168.60.20:/backups /mnt`	`/etc/fstab`
an IP address	`ip addr add …`	`/etc/netplan/…yaml`
a saved file	– (saving in /home, /etc … already goes to disk)	`any folder on the disk = stays`

admin habit: ① try it LIVE → ② check it works → ③ write it into the FILE so it stays

/proc /sys /dev /run = see and change the system live (kernel views + RAM) – good for trying first · /etc = the settings programs read at start

B · The test pattern – systemd-run makes a temporary box, -p puts a limit on it, stress-ng makes fake load

```
sudo systemd-run --scope -p <LIMIT> stress-ng <LOAD> --timeout <TIME>
```

`--scope`
box for MY command: output on my screen, deleted when the command ends

`-p <LIMIT>`
MemoryMax = kill · MemoryHigh = slow + swap
CPUQuota = throttle · TasksMax = max processes

`stress-ng <LOAD>`
`--vm 1 --vm-bytes 300M` = 1 worker 300MB
`--cpu 1` = busy core · `--fork 4` = forks

where does the box go?
with sudo → the system systemd → system.slice (the default place)

C · My test on app01: MemoryMax=100M, no swap, stress-ng wants 300M → killed inside its box only

1. **systemd-run** — temporary box — `run-ra9665….scope`
2. **stress-ng worker** — wants ~340 MB — `total-vm 349924kB`
3. **hits the wall** — real RAM ~96 MB — `anon-rss 98792kB`
4. **OOM killer** — kills the worker — `7× · new PID each time`
5. **parent survives** — PID 2687 — `"successful run completed"`
6. **box deleted** — command ended — `nothing left behind`

proof in the journal: `constraint=CONSTRAINT_MEMCG · oom_memcg=/system.slice/run-ra9665….scope`

CONSTRAINT_MEMCG = killed because of the BOX limit, not because app01 ran out of RAM · only that box was touched – services next to it kept running
same idea as a Kubernetes pod "OOMKilled" or Docker --memory

D · Useful checks

`pgrep -a stress-ng`	is a test still running?
`systemctl list-units --all | grep mytest`	old named test boxes?
`sudo pkill stress-ng · systemctl stop mytest`	stop leftovers
`sudo systemctl reset-failed`	clean old failed boxes
`sudo journalctl -k --since "5 min ago" | grep -i killed`	OOM kills in the kernel log
`findmnt · free -h · cat /etc/fstab`	mounts · RAM/swap · mount recipe

Final paragraph – the whole day in one place

Everything is stored on the disk, and RAM starts empty. At power-on the kernel (vmlinuz with its helper initrd.img) is copied from /boot into RAM and becomes the manager; it starts systemd as PID 1. systemd reads its recipes in /etc – each read puts a copy of that one file in RAM (page cache) while the original stays on the disk – and from them builds the live things in RAM: the mount table, the services and the cgroup boxes. Every program asks the kernel for files; the kernel gives a copy from RAM, or reads the disk first. Each folder is mounted to a source: disk folders stay, /run in RAM is gone at reboot, /proc /sys /dev are live kernel information, the NFS share stays on stor01. The work always happens in RAM, and WHERE I write it decides if it stays: a live change (sysctl -w, systemd-run, --runtime, mount, ip addr) is a test that is gone at reboot; a file on the disk – usually in /etc – is read at every boot and stays. My test proved it: systemd-run made a temporary box in system.slice with MemoryMax=100M, the OOM killer stopped stress-ng at about 96 MB inside that box only (CONSTRAINT_MEMCG), and the box disappeared when the test ended.

Phase 4 limits · page 1 – my 3 limit tests on app01

same pattern every time: sudo systemd-run --scope -p <LIMIT> stress-ng <LOAD> → watch what the kernel does

make a BOX	→	put a LIMIT on it	→	run fake LOAD inside	→	read the PROOF
`systemd-run --scope`		`-p MemoryMax / MemoryHigh / CPUQuota`		`stress-ng --vm / --cpu`		`journalctl · top · cpu.max`

TEST 1 – hard memory limit

limit: **`MemoryMax=100M MemorySwapMax=0`**

load: `stress-ng --vm 1 --vm-bytes 300M`

(fake program to test the limit)

What happened

- wants ~340 MB
- reaches ~96 MB = the wall
- no swap allowed → nowhere to go

KILLED

by the OOM killer (7× – new PID each time)

Proof I saw

```
journalctl -k … | grep -i -E "oom|killed"
"Killed process … CONSTRAINT_MEMCG"
parent survived: "successful run"
```

like a full glass: no more fits → thrown out

remember: MemoryMax = KILL

memory has no "wait" – it must free space

TEST 2 – soft memory limit

limit: **`MemoryHigh=100M (swap allowed)`**

load: `stress-ng --vm 1 --vm-bytes 200M`

(fake program to test the limit)

What happened

- wants 200 MB
- above 100 MB: kernel pushes
- the extra into SWAP on disk

SLOWED

not killed – finished in 22.17 s, not 20 s

Proof I saw

```
journalctl -k … | grep -i killed
→ prints NOTHING
"successful run completed in 22.17s"
```

like a full desk: extra papers go to the shelf

remember: MemoryHigh = SLOW

kernel moves memory to disk instead

TEST 3 – CPU limit

limit: **`CPUQuota=20%`**

load: `stress-ng --cpu 1`

(fake program to test the limit)

What happened

- no limit: 100 % (one full core)
- with limit: may run 20 ms
- in every 100 ms, then WAITS

THROTTLED

top: 100 % → 20.5 % · never killed

Proof I saw

```
top → stress-ng %CPU 20.5
cpu.max → "20000 100000"
2 boxes → each got its own 20 %
```

like mobile data after the limit: slower, not cut

remember: CPUQuota = WAIT

CPU time can always be given later

Page 2 – when the hard wall is hit: who is killed, and how to read the proof

test 1 continued · the OOM score · protecting services · what systemd does after · console tricks

A · 6 services share one box (MemoryMax=1G) – total 1070 MB → over

web uses 120 MB · adj 0 **survives**	**db** uses 380 MB · adj 0 **KILLED – biggest user**	**cache** uses 150 MB · adj 0 **survives**
sshd uses 10 MB · adj -1000 **protected – never**	**worker** uses 90 MB · adj 0 **survives**	**6th: report** uses 300 MB · adj 0 **survives**

the 6th service pushed the box over – but "db" dies: the kernel kills the HIGHEST SCORE, mostly the biggest memory user – predictable, not luck, not "the newest one"

B · Who does what

1	KERNEL	sees the box hit MemoryMax
2	KERNEL	scores every process in THIS box
3	KERNEL	kills the highest score
4	systemd	marks: failed (Result: oom-kill)
5	systemd	restarts ONLY if Restart= is set
6	systemd	OOMPolicy=stop → ends whole service
		OOMPolicy=continue → rest keeps running

systemd does NOT choose – the kernel does

C · The OOM score – final score = memory part (kernel measures) + oom_score_adj (I set: -1000 … +1000)



sshd: 0
adj -1000

my bash: 667
adj 0 · normal

~1333
all RAM

checked on app01:

`cat /proc/$$/oom_score`	**667**
`cat /proc/$$/oom_score_adj`	**0**
`pgrep -o sshd`	**966**
`…/$(pgrep -o sshd)/oom_score_adj`	**-1000**

protect an important service: `OOMScoreAdjust=-1000 (or -500) in its unit file · live test: echo -500 | sudo tee /proc/<PID>/oom_score_adj`

make one die first: `OOMScoreAdjust=+1000 (stress-ng sets this on itself – that is why it was picked)`

careful · best design: **protect only the few that matter (if all are protected nothing can be freed) · best: one box per service**

D · Reading the proof – 3 commands joined by pipes (the output of one goes into the next)

```
sudo journalctl -k --since "5 min ago" | grep -i -E "oom|killed" | tail -5
```

`journalctl -k --since "5 min ago"` the kernel's messages, last 5 min	→	`grep -i -E "oom|killed"` only lines with oom OR killed (any case)
→		`tail -5` only the newest 5 lines

E · Console tricks

`cmd & sleep 2; top`	& = background · sleep 2 = let it start · ; = then · top = watch it live
`sudo -v`	type the password first, so a background sudo command is not stuck waiting
`…/system.slice/run-*.scope/cpu.max`	a box folder exists only WHILE its test runs · slash, not dash · Tab completes paths
`Alt+F1 / Alt+F2 · sudo chvt 2`	switch between console 1 and 2 (click inside the VM first)

Final paragraph – Phase 4 limits, the whole evening: one pattern runs every test – systemd-run --scope makes a temporary box, -p puts a limit on it, stress-ng loads it, and I read the proof. MemoryMax is a hard wall: the job is KILLED (test 1, ~96 MB, CONSTRAINT_MEMCG). MemoryHigh is a soft limit: the job is SLOWED into swap (test 2, 22.17 s, no kill). CPUQuota makes the box WAIT – 20 ms of every 100 ms (test 3, 100 % → 20.5 %, cpu.max 20000 100000); each box has its own limit. When a shared box hits its wall, the KERNEL scores every process in that box – memory used plus oom_score_adj – and kills the highest, usually the biggest user, not the newest. On app01 my bash scores 667 (normal) and sshd is protected at -1000, so I can always log in. systemd only reacts: it marks the service failed (oom-kill), restarts it only if Restart= is set, and OOMPolicy decides whether the rest of that service stops or continues. I can protect a service with OOMScoreAdjust=-1000 in its unit file; the best design is one box per service. MemoryMax = kill · MemoryHigh = slow · CPUQuota = wait.

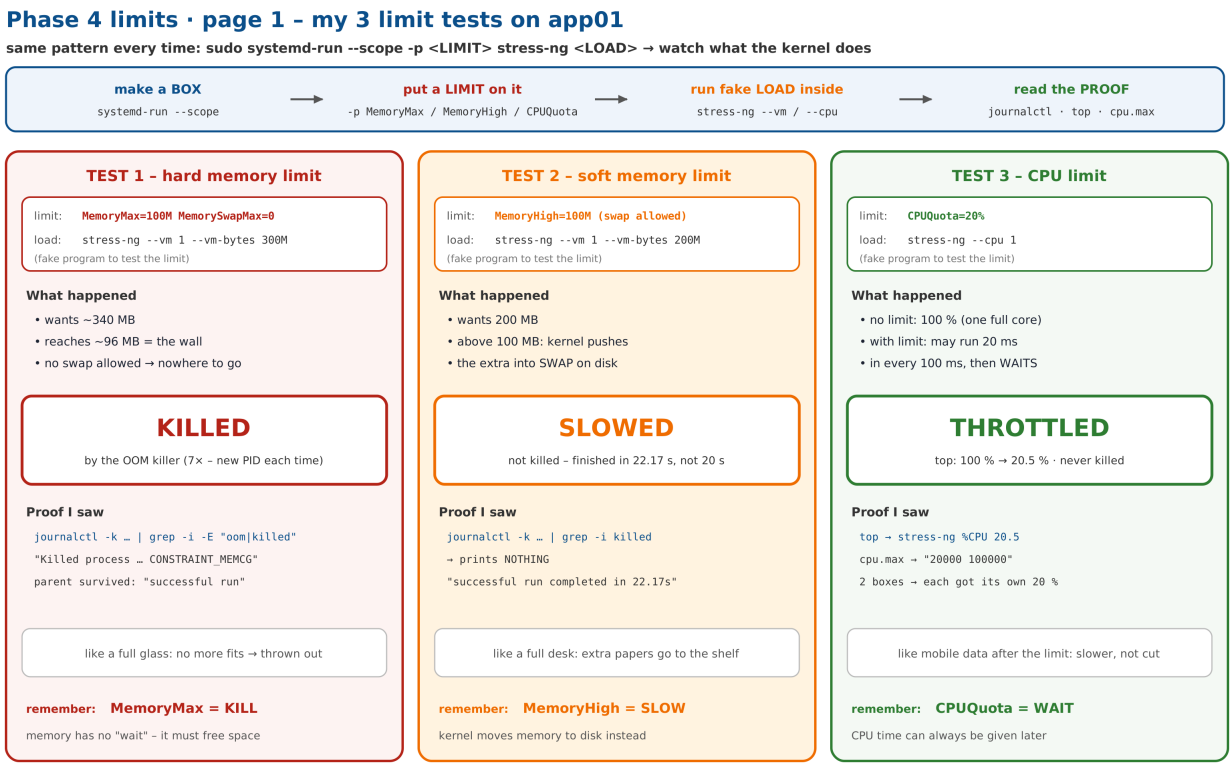
Step 4 on stor01 – permanent limits for archive-worker

Phase 4 · 4 November 2026 · every command with its own screenshot · refresher of tests 1–3 first

Refresher – what tests 1–3 taught (read this first)

	limit	what happens	remember
Test 1	`MemoryMax=100M`	hard wall – the OOM killer stops the job inside the box	**KILL**
Test 2	`MemoryHigh=100M`	soft limit – extra memory pushed to swap, job slower, not killed	**SLOW**
Test 3	`CPUQuota=20%`	box may run 20 ms of every 100 ms, then waits – throttled	**WAIT**

Tests 1–3 used `systemd-run --scope` = a **temporary** box, a **live change** in RAM – gone when the test ended. Today the limits go on a **real service** and into a **file on the disk** – the "recipe" that systemd reads at every boot. That is what makes them permanent.



Today's plan – 5 steps

Step 4 on stor01 – permanent limits for archive-worker.service

tests 1–3 were LIVE (gone after the test) · now the limits go into a FILE on the disk → they survive every reboot

1 **CHECK** — what limits does it have now? `systemctl show …`

2 **SET** — set-property writes a FILE `CPUQuota=30% …`

3 **SEE THE FILE** — the drop-in on the disk `systemctl cat …`

4 **LIVE CHECK** — the box got the limits `/sys/fs/cgroup/…`

5 **REBOOT TEST** — still there after reboot? `sudo reboot → show`

The 4 limits for the archive worker

`CPUQuota=30%`	max 30 % of one core → WAIT (test 3)
`MemoryHigh=150M`	above 150 MB → SLOWED into swap (test 2)
`MemoryMax=200M`	at 200 MB → KILLED (test 1)
`TasksMax=20`	max 20 processes → new forks fail

plus Nice=19 already set on Day 8 = priority (who goes first)

Why it is permanent – the recipe idea

FILE on disk `/etc/systemd/system.control/` → live box in RAM `/sys/fs/cgroup/…`

- set-property writes the file AND applies it at once
- every boot: systemd reads the file → builds the box
- **no --runtime this time (that would go to /run = RAM)**

the box of a service lives here: /sys/fs/cgroup/system.slice/archive-worker.service/ (cpu.max · memory.high · memory.max · pids.max)

Final paragraph: Tests 1 to 3 were live changes – the temporary boxes and their limits disappeared when each test ended. For a real service the limits must survive a reboot, so they go into a file. sudo systemctl set-property archive-worker.service CPUQuota=30% MemoryHigh=150M MemoryMax=200M TasksMax=20 does two things at once: it applies the limits to the running box and writes them as a drop-in file under /etc/systemd/system.control/ on the disk. At every boot systemd reads that file and builds the box again with the same limits. I check the file with systemctl cat, the live box under /sys/fs/cgroup/system.slice/archive-worker.service/, and prove it is permanent with a reboot. CPUQuota makes the worker wait, MemoryHigh slows it, MemoryMax kills it, TasksMax stops it creating too many processes – and Nice=19 from Day 8 still decides that it goes last.

Step 1 – check what the worker has now

```
systemctl status archive-worker --no-pager
```

My Computer | gw01 | app01 | **stor01** | svc01

```
bilal_003@stor01:~$ systemctl status archive-worker --no-pager
● archive-worker.service - Acme archive worker (processes the archive queue)
     Loaded: loaded (/etc/systemd/system/archive-worker.service; enabled; vendor preset: enabled)
     Active: active (running) since Wed 2026-11-04 14:36:57 UTC; 2min 34s ago
   Main PID: 1019 (archive-worker.)
      Tasks: 2 (limit: 964)
     Memory: 1.0M
        CPU: 689ms
     CGroup: /system.slice/archive-worker.service
             ├─1019 /bin/bash /usr/local/bin/archive-worker.sh
             └─1639 sleep 5

Nov 04 14:36:57 stor01 systemd[1]: Started Acme archive worker (processes the archive queue).
Nov 04 14:36:57 stor01 archive-worker[1024]: worker started (PID 1019)
```

What it shows: the worker is **active (running)** since boot (enabled = starts automatically). Its box is `/system.slice/archive-worker.service` with 2 processes inside: the worker script (PID 1019) and the `sleep 5` it runs while waiting for jobs. **Tasks: 2 (limit: 964)** – 964 is the default, not a limit I chose. Memory 1.0 MB – tiny.

```
systemctl show archive-worker -p CPUQuotaPerSecUSec -p MemoryHigh -p MemoryMax -p
TasksMax -p Nice
```

```
bilal_003@stor01:~$ systemctl show archive-worker -p CPUQuotaPerSecUSec -p MemoryHigh -p MemoryMax -
p TasksMax -p Nice
CPUQuotaPerSecUSec=infinity
MemoryHigh=infinity
MemoryMax=infinity
TasksMax=964
Nice=19
```

Before: `infinity` = no CPU or memory limit · TasksMax 964 = the default · `Nice=19` = the low priority I set on Day 8. Priority says *who goes first*; it does not cap anything.

Step 2 – set the 4 limits, permanently

```
sudo systemctl set-property archive-worker.service CPUQuota=30% MemoryHigh=150M
MemoryMax=200M TasksMax=20
```



No output = success. **No `--runtime`** this time – with it the limits would go to `/run` (RAM) and vanish at reboot. Without it, set-property does two things at once: **applies** the limits to the running box and **writes** them to files on the disk.

limit	meaning for the worker
`CPUQuota=30%`	max 30 % of one core – a big archive job cannot take the CPU from NFS or backups (test 3 = wait)
`MemoryHigh=150M`	above 150 MB it is slowed into swap (test 2)
`MemoryMax=200M`	at 200 MB the OOM killer stops it (test 1) – `Restart=on-failure` in its unit file then starts it again
`TasksMax=20`	max 20 processes – a bug that forks endlessly cannot fill the PID pool

Step 3 – see the files on the disk (the recipe)

```
systemctl cat archive-worker
```

```
bilal_003@stor01:~$ systemctl cat archive-worker
# /etc/systemd/system/archive-worker.service
[Unit]
Description=Acme archive worker (processes the archive queue)
After=local-fs.target
[Service]
ExecStart=/usr/local/bin/archive-worker.sh
Restart=on-failure
Nice=19
IOSchedulingClass=idle
[Install]
WantedBy=multi-user.target
```

Part 1 – my original unit file `/etc/systemd/system/archive-worker.service` from Day 8 – unchanged: the script, `Restart=on-failure`, `Nice=19`, `IOSchedulingClass=idle`.

```
# /etc/systemd/system.control/archive-worker.service.d/50-CPUQuota.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
CPUQuota=30%

# /etc/systemd/system.control/archive-worker.service.d/50-MemoryHigh.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
MemoryHigh=157286400

# /etc/systemd/system.control/archive-worker.service.d/50-MemoryMax.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
MemoryMax=209715200

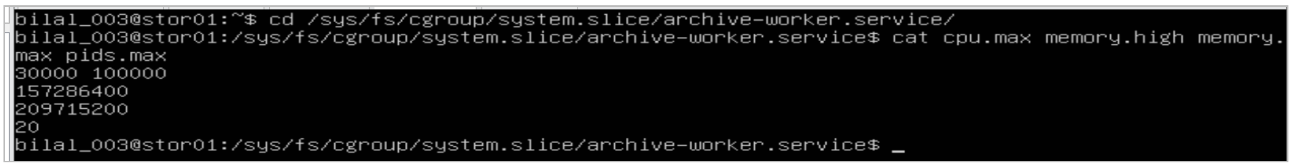
# /etc/systemd/system.control/archive-worker.service.d/50-TasksMax.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
TasksMax=20
bilal_003@stor01:~$ _
```

Part 2 – four new drop-in files in `/etc/systemd/system.control/archive-worker.service.d/`, one per limit (`50-CPUQuota.conf` …). They are added *on top of* the unit file without changing it. "Do not edit" – to change a limit, run set-property again.

Memory is stored in **bytes**: 157286400 = 150 × 1024 × 1024 = 150 MB · 209715200 = 200 MB.

Step 4 – check the live box in the kernel

```
cd /sys/fs/cgroup/system.slice/archive-worker.service/
cat cpu.max memory.high memory.max pids.max
```

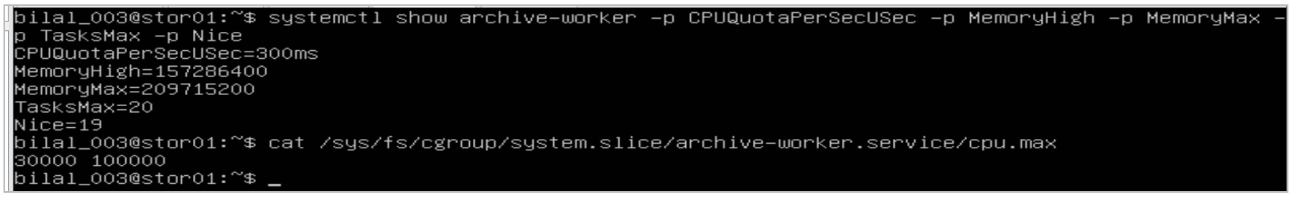


file	value	meaning
`cpu.max`	30000 100000	30 ms of every 100 ms = 30 %
`memory.high`	157286400	150 MB soft limit
`memory.max`	209715200	200 MB hard wall
`pids.max`	20	max 20 processes

The file on the disk (step 3) is the **recipe**; these values in `/sys/fs/cgroup` are the **live box** in RAM that the kernel enforces.

Step 5 – the reboot test (proof that it is permanent)

```
sudo reboot
# after logging in again:
systemctl show archive-worker -p CPUQuotaPerSecUSec -p MemoryHigh -p MemoryMax -p
TasksMax -p Nice
cat /sys/fs/cgroup/system.slice/archive-worker.service/cpu.max
```



after reboot	meaning
`CPUQuotaPerSecUSec=300ms`	30 % = 300 ms of CPU per second – same limit, new way of writing it
`MemoryHigh=157286400` · `MemoryMax=209715200`	150 MB soft · 200 MB hard – still there
`TasksMax=20` · `Nice=19`	process cap and priority – still there
`cpu.max 30000 100000`	the kernel's live box was rebuilt with the limit

Proof: RAM was emptied by the reboot, yet the box came back with every limit – because systemd read the drop-in files on the disk at boot and built the box again. ✅ Permanent.

Final paragraph: Tests 1 to 3 were live changes – their temporary boxes and limits disappeared when each test ended. For the real archive worker on stor01 the limits must survive a reboot, so they go into files. Before, the worker had no CPU or memory limit

(`infinity`), the default TasksMax 964 and Nice=19 from Day 8. `sudo systemctl set-property archive-worker.service CPUQuota=30% MemoryHigh=150M MemoryMax=200M TasksMax=20` – without `--runtime` – applied the limits at once and wrote four drop-in files under `/etc/systemd/system.control/archive-worker.service.d/` , next to my unchanged unit file; `systemctl cat` shows both. The kernel's live box in `/sys/fs/cgroup/system.slice/archive-worker.service/` confirms `cpu.max 30000 100000` , memory.high 150 MB, memory.max 200 MB and pids.max 20. Now the worker waits above 30 % CPU, is slowed above 150 MB, is killed at 200 MB – and restarted by `Restart=on-failure` – and can never run more than 20 processes, while Nice=19 still makes it go last. At every boot systemd reads the drop-ins and builds the box again: the reboot test proved it: after the restart `systemctl show` still reported 300ms, 150 MB, 200 MB, 20 tasks and Nice=19, and cpu.max was again 30000 100000.

Step 4 explained – how the 4 limits really work

questions from revising: how does CPUQuota=30% work in time? what does TasksMax count? how are restarts limited?

CPU throttling – CPUQuota=30% on stor01, everything on one page

one CPU, three jobs: the archive worker · the NFS share for app01 · the nightly backup (or any other job running in the background)

① Who gets the CPU

WITHOUT a limit

archive job 92%

the archive job takes ~92 %

app01 waits for NFS · the backup crawls

the others only get tiny gaps

WITH CPUQuota=30%

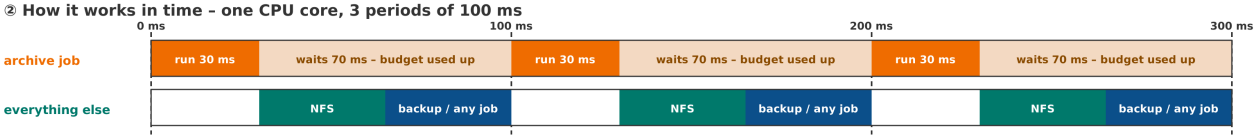
archive 30% | NFS 35% | backup / any job 35%

the archive job never gets more than 30 %

NFS and the backup keep their share

the archive job just takes longer

② How it works in time – one CPU core, 3 periods of 100 ms



refill every 100 ms BY THE CLOCK → run · wait · run · wait … 10 times per second

③ The facts to remember

`cpu.max = 30000 100000`
30 000 µs (30 ms) budget in every 100 000 µs (100 ms) period

runs at FULL speed
until its 30 ms are used up – the CPU never slows down

then it WAITS
only until the next 100 ms period – not until the backup ends

the 70 ms go to
whatever else needs the CPU: NFS, backup, sshd, any job

nothing else running?
the CPU stays idle – CPUQuota is a HARD cap (30 % stays 30 %)

never killed
CPU can always be given later – too much CPU only waits

④ Pocket money that refills every "day"

every day you get 300 (= 30 ms every 100 ms)
you spend at your NORMAL speed until it is gone
then you have 0 → you wait until tomorrow
tomorrow: 300 again → spend → wait → …

300	wait	300	wait	300	wait
day 1 = 100 ms		day 2 = 100 ms		day 3 = 100 ms	

the speed of spending never changes – only the waiting

⑤ Proven in test 3 (app01): no limit → stress-ng 100 % · CPUQuota=20% → 20.5 % · cpu.max 20000 100000 · never killed

and on stor01 (Step 4): archive-worker cpu.max 30000 100000 – still there after reboot

Final paragraph: On stor01 the archive worker, the NFS share for app01 and the nightly backup share one CPU. Without a limit one big archive job could take about 92 % of it, so app01 would wait for NFS and the backup would crawl. CPUQuota=30% gives the archive box a budget of 30 ms in every 100 ms period (cpu.max 30000 100000). The job runs at its normal full speed until the 30 ms are used up, then it must wait – not until the backup ends, only until the next 100 ms period, when the clock refills its budget. In the other 70 ms the CPU works for whatever else needs it: NFS, the backup or any other job; if nothing else is running the CPU stays idle, because CPUQuota is a hard cap. Like pocket money paid every day, the spending speed never changes – only the waiting. This repeats 10 times per second, so the archive job gets 30 % on average and simply takes longer; nothing is killed. Test 3 proved it (100 % → 20.5 % with 20 %), and on stor01 the 30 % limit survived a reboot.

Two lines made clear – who shares stor01's CPU, and what TasksMax really counts

① "A big archive job cannot take the CPU from NFS and backups"

NFS (not NFC) = the network share on stor01 · all three jobs run ON stor01 and share ITS CPU – it is not about the CPU of another server

WITHOUT a limit

stor01 CPU (100 %)

archive job 92%

the archive job takes almost everything

→ app01 waits for the NFS share (slow)

→ the nightly backup runs very slowly

(backup + NFS fight for the last few %)

WITH CPUQuota=30%

stor01 CPU (100 %)

archive 30% | NFS 35% | backup 35%

the archive job may use at most 30 %

→ NFS keeps answering app01 quickly

→ the backup still gets its share

(the archive job just takes longer – fine)

② TasksMax is NOT "how many restarts" – it is "how many processes at the SAME time"

TasksMax=20

max 20 processes/threads ALIVE at once in the box

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20

the 21st fork fails: "Resource temporarily unavailable"

protects stor01's PID pool from a fork bug

restarts – a DIFFERENT setting

`Restart=on-failure` → restart it when it crashes / is killed

`StartLimitBurst=5` → at most 5 starts …

`StartLimitIntervalSec=10s` … within 10 seconds

more often → systemd gives up: "start request repeated too quickly" – the service stays failed

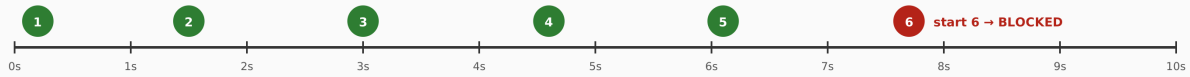
(5 in 10 s are the systemd defaults)

Final paragraph: stor01 does three jobs on the same CPU – it shares /backups over NFS to app01, it runs the nightly backups, and it runs the archive worker. Without a limit one big archive job could take almost all of stor01's CPU, so app01 would wait for the NFS share and the backups would crawl. CPUQuota=30% lets the worker use at most 30 % of a core, so NFS and the backups always keep their share; the archive job only takes longer. TasksMax=20 is something else: it is the maximum number of processes or threads alive in the worker's box at the same time – the 21st fork fails, which protects the PID pool. It does not count restarts. Restarts are set by Restart=on-failure (restart when it dies) and limited by StartLimitBurst and StartLimitIntervalSec (by default 5 starts in 10 seconds, then systemd gives up and the service stays failed).

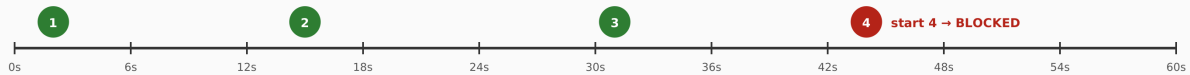
StartLimitBurst + StartLimitIntervalSec – the brake on endless RESTARTS (not threads)

one rule: "at most BURST starts within INTERVAL seconds" – two examples with different numbers

Example A – the DEFAULT (what every service has if I set nothing): 5 starts in 10 seconds



Example B – MY OWN choice in the unit file: 3 starts in 60 seconds



Example B written in the unit file

```
[Unit]
StartLimitIntervalSec=60 ← INTERVAL: 60 s
StartLimitBurst=3 ← BURST: 3 starts

[Service]
Restart=on-failure ← restart when it dies
```

Check and recover

```
systemctl show archive-worker -p StartLimitBurst \
   -p StartLimitIntervalUSec
```

archive-worker has nothing set → shows the default 5 · 10s

blocked? fix the cause, then:

```
sudo systemctl reset-failed archive-worker
```

Final paragraph: Restart=on-failure starts a service again when it dies. To stop an endless crash–restart loop, systemd counts the STARTS (not threads or processes) inside a time window: StartLimitBurst is how many starts are allowed, StartLimitIntervalSec is the length of the window. If I set nothing, the default is 5 starts in 10 seconds (example A); in the unit file I can choose my own numbers, for example StartLimitIntervalSec=60 and StartLimitBurst=3 = at most 3 starts per minute (example B). One start too many is blocked with "start request repeated too quickly" and the service stays failed until I fix the cause and run systemctl reset-failed. TasksMax is different: it counts processes running at the same moment.

Incident drill on stor01 – "I deleted the file, but the disk is still full"

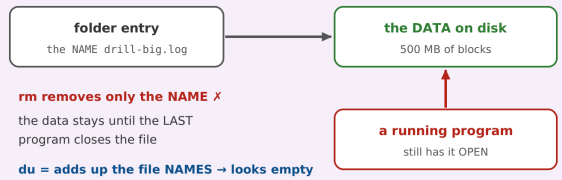
Phase 4 · 4 November 2026 · a deleted file still held open by a running program · every command with its own marked-up screenshot

Incident drill on stor01 – "I deleted the file, but the disk is still full"

a classic real ticket and interview question: a program still holds a deleted file open, so its space is NOT freed

Ticket (client words): "/backups on stor01 was nearly full. I deleted a big old log file – but df still says it is full!"
real-life cause: logrotate or a person deletes a log file while the program that writes it keeps running

Why the space is not freed



rm removes only the NAME ✗

the data stays until the LAST program closes the file

du = adds up the file NAMES → looks empty
df = counts the data blocks → still FULL

The drill – 4 parts

1 BUILD the problem
make a 500 MB file · a program holds it · delete it

2 SEE the symptom
df: still used · du: nothing there

3 FIND the cause
sudo lsof +L1 → "(deleted)" + the PID

4 FIX + TICKET
stop that program → space free · GLPI ticket

How admins fix it in real life

- best: restart the service that holds the file (systemctl restart …) – it closes the old file and opens a new one
- if it must not restart: empty the deleted file through the process: : > /proc/<PID>/fd/<FD> – frees the space at once

Final paragraph: In Linux, deleting a file with rm only removes its name from the folder. The data blocks are freed only when the last program that has the file open closes it. So if a running program still holds a deleted file – typically a log file removed while the service keeps writing – df still counts the space as used, while du, which adds up the file names it can see, shows nothing. The tool to find it is sudo lsof +L1: it lists open files whose name is gone, marked "(deleted)", with the PID and file descriptor of the program holding them. The fix is to restart or stop that program (or empty the file through /proc/<PID>/fd/<FD> if it must keep running), and the space comes back at once. In the drill I build this on stor01's /backups, find it, fix it and record it as a GLPI ticket.

The incident drill – final corrected version, in plain words

an INCIDENT drill (a disk-space mystery), not a restore drill · /backups was used only because it is a separate disk with easy numbers

1 look at the disk — `df -h /backups`
how full is the /backups disk? → 60K used

2 make a big file — `sudo fallocate -l 500M …`
fallocate = FILE ALLOCATE = reserve 500 MB for one empty file (no stress, just space) → 501M used

3 a process holds it OPEN — `sleep 3600 < file &`
the KEY step: process sleep (PID 5468) holds the file open on its file descriptor 0 – it plays a service and its log

4 delete the name — `sudo rm file`
only the NAME is gone – USED stays 501M, because the process still holds the file open

5 prove the mystery — `sudo du -sh /backups`
du = disk usage · -s = summary (one total) · -h = human-readable → only 56K (du counts names)

6 find + fix — `sudo lsof +L1 → kill 5468`
lsof = list open files · +L1 = less than 1 name left = deleted but still open → shows the PID of the PROCESS → stop it → 60K

Words – say it right

fallocate	file ALLOCATE (reserve), not "locate"
the PROCESS has the PID	the file has no PID
process HOLDS it open	a file has no "parent" – that word is for processes
df vs du	df counts disk blocks · du counts file names

Same effect with a copy A → B

cp is reading file X · someone deletes X
→ cp still finishes the copy · X's space stays used
→ freed when cp closes X – after seconds, nobody notices

a SERVICE keeps its log open for weeks
→ the space stays "lost" until the service is restarted

Final paragraph: This was an incident drill, not a restore drill. df -h /backups showed the disk almost empty (60K). sudo fallocate (file allocate) reserved 500 MB for one empty file – no stress, just space – so used went to 501M. The key step: sleep 3600 < file & – a process (PID 5468) holding the file open on its file descriptor 0, like a service holds its log. sudo rm removed only the name, so used stayed at 501M, while du -sh (disk usage, summary, human-readable) counted only 56K, because du adds up the names it can find and df counts the blocks. lsof +L1 (list open files with fewer than one name) showed the deleted file and the PID of the process holding it – the file itself has no PID and no parent. Stopping that process freed the 500 MB; on a real server I would restart the service. The same happens when a file is deleted while cp is copying it, but a copy closes the file within seconds – a service keeps its log open for weeks, which is why this becomes a real incident.

1 – Build the problem

```
df -h /backups
```



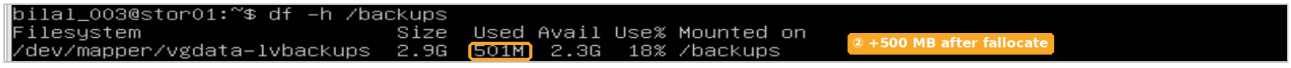
① **Before:** /backups (an LVM volume, 2.9 GB) uses only **60K** – it is empty, 1 %.

```
sudo fallocate -l 500M /backups/drill-big.log
```



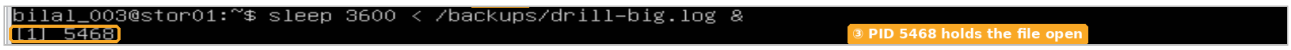
No output = success. `fallocate` reserves 500 MB on the disk at once – our big "log file".

```
df -h /backups
```



② **After fallocate: 501M** used, 18 % – the new file takes the 500 MB.

```
sleep 3600 < /backups/drill-big.log &
```



③ **A program holds the file open:** `sleep` uses the file as its input and runs for an hour in the background – job **[1]**, **PID 5468**. It stands in for a real service keeping its log file open.

```
sudo rm /backups/drill-big.log
```



No output = the file name is deleted.

2 – See the symptom

```
df -h /backups
```



④ **The problem:** still **501M** used after `rm`! `df` counts the data blocks on the disk – they are still there.

```
sudo du -sh /backups
```



⑤ **The contradiction:** `du` adds up the file names it can find – the name is gone, so it sees only **56K**. When df and du disagree, think "deleted but still open".

3 – Find the cause

```
sudo lsof +L1
```



⑥ **The cause:** `lsof +L1` lists open files that have no name left (fewer than 1 link). Marked: **PID 5468** (sleep) · **524288000** bytes = 500 MB · **(deleted)**. FD `0r` = its input, opened for reading · NLINK `0` = no names left.

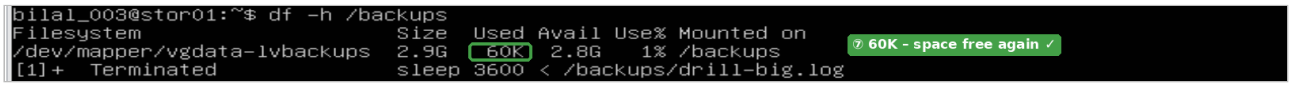
4 – Fix it

```
kill 5468
```



Stop the program that holds the file (the PID from lsof).

```
df -h /backups
```



⑦ **Fixed:** back to **60K**, 1 % – the kernel freed the 500 MB as soon as the last program closed the file. `[1]+ Terminated` confirms the background job ended.
On a real server I would not kill a service blindly – I would **restart** it (`sudo systemctl restart <service>`) so it closes the old log and opens a new one; if it must keep running, `: > /proc/<PID>/fd/<FD>` empties the deleted file in place.

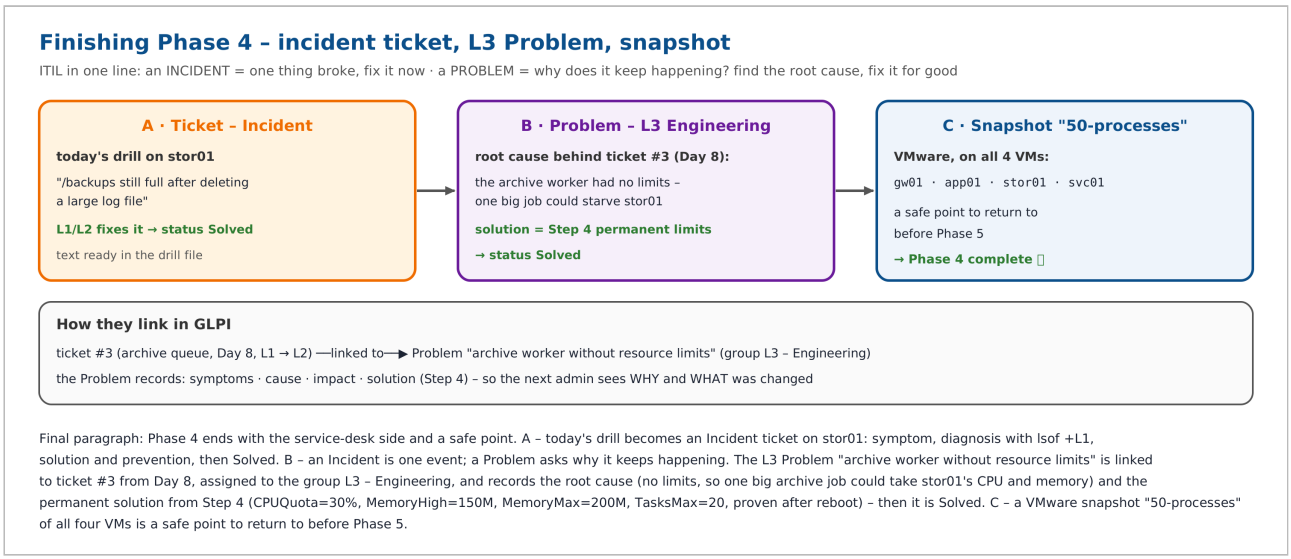
5 – Record it as a GLPI ticket (text ready to paste)

Title: stor01 – /backups still full after deleting a large log file
Type: Incident · **Category:** Storage · **Urgency:** Medium · **Asset:** stor01
Description: User deleted /backups/drill-big.log (500 MB) but `df -h /backups` still shows 501M used (18 %). `du -sh /backups` shows only 56K.
Diagnosis: `sudo lsof +L1` → process `sleep`, PID 5468, holds /backups/drill-big.log (deleted), 524288000 bytes, NLINK 0. The space is not freed while a process keeps the deleted file open.
Solution: Stopped the holding process (kill 5468). `df -h /backups` → 60K used, 1 %. In production: restart the service that holds the file, or empty it via /proc/<PID>/fd/<FD>.
Prevention: rotate logs with logrotate using `copytruncate` or a service reload, never delete open log files by hand; check `lsof +L1` when df and du disagree.

Final paragraph: This was an incident drill, not a restore drill. `df -h /backups` showed the disk almost empty (60K). `sudo fallocate` (file allocate) reserved 500 MB for one empty file – no stress, just space – so used went to 501M. The key step: `sleep 3600 < file &` – a process (PID 5468) holding the file open on its file descriptor 0, like a service holds its log. `sudo rm` removed only the name, so used stayed at 501M, while `du -sh` (disk usage, summary, human-readable) counted only 56K, because du adds up the names it can find and df counts the blocks. `lsof +L1` (list open files with fewer than one name) showed the deleted file and the PID of the process holding it – the file itself has no PID and no parent. Stopping that process freed the 500 MB; on a real server I would restart the service, and prevent it with proper log rotation. The same happens when a file is deleted while `cp` is copying it, but a copy closes the file within seconds – a service keeps its log open for weeks, which is why this becomes a real incident. The incident is recorded in GLPI as ticket #4.

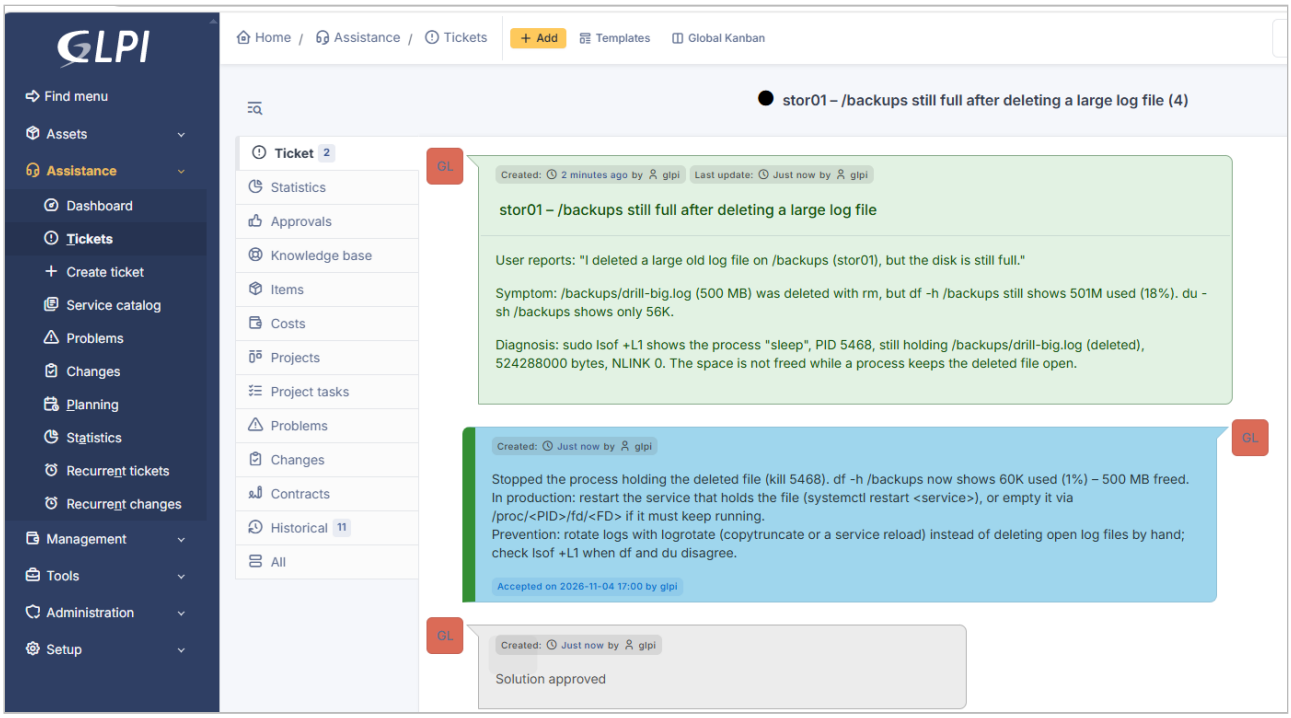
Closing Phase 4 – Incident ticket, L3 Problem, snapshot

4 November 2026 · GLPI on svc01 (http://192.168.60.40) · ITIL: Incident = fix it now · Problem = find the root cause, fix it for good



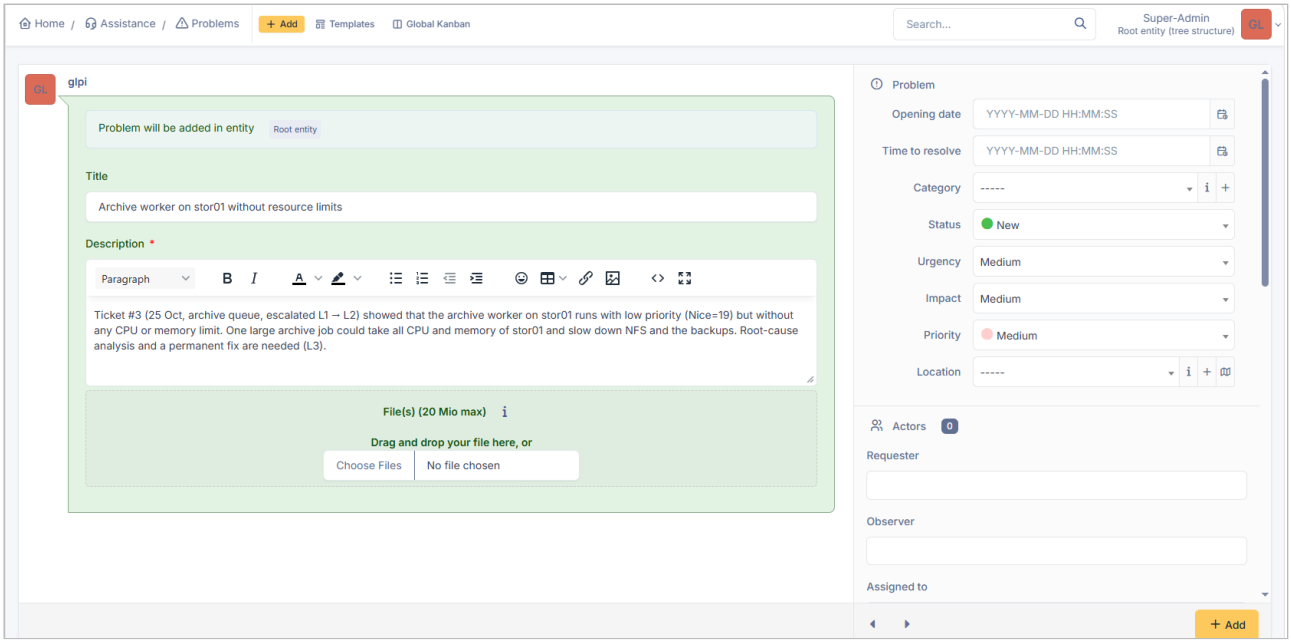
A – Incident ticket #4 for today's drill – Solved and closed

Title *stor01 – /backups still full after deleting a large log file* · Type Incident · Category Storage & Backup · Requester **alice** (Requester = who reported it) · Assigned to group **L2 – Linux Storage** · Item **stor01** (Item = what is broken). The description holds the symptom and the lsof diagnosis; the solution holds the fix and the prevention. The solution was accepted and the ticket closed (black dot). In a company alice would approve it herself.

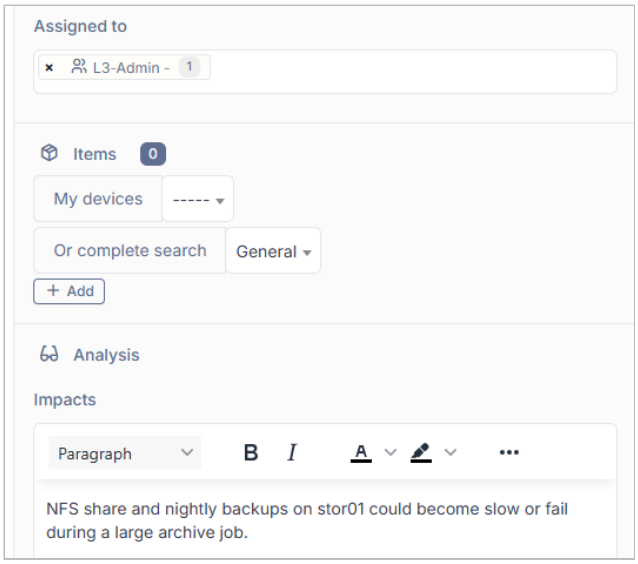


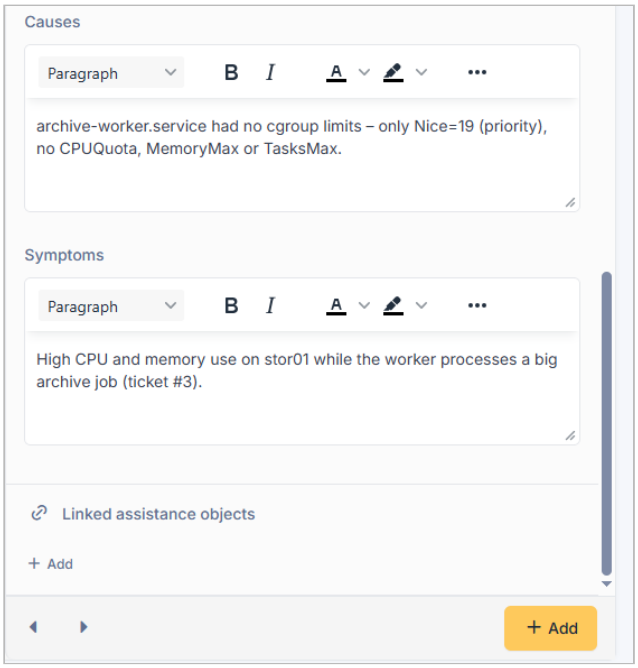
B – L3 Problem: "Archive worker on stor01 without resource limits"

An Incident is one event; a Problem asks why it happens. This Problem is the root cause behind **ticket #3** (25 Oct, archive queue, escalated L1 → L2). Assigned to group **L3-Admin**. Note: it must be created under **Assistance → Problems** – the same text under Tickets would create a ticket.



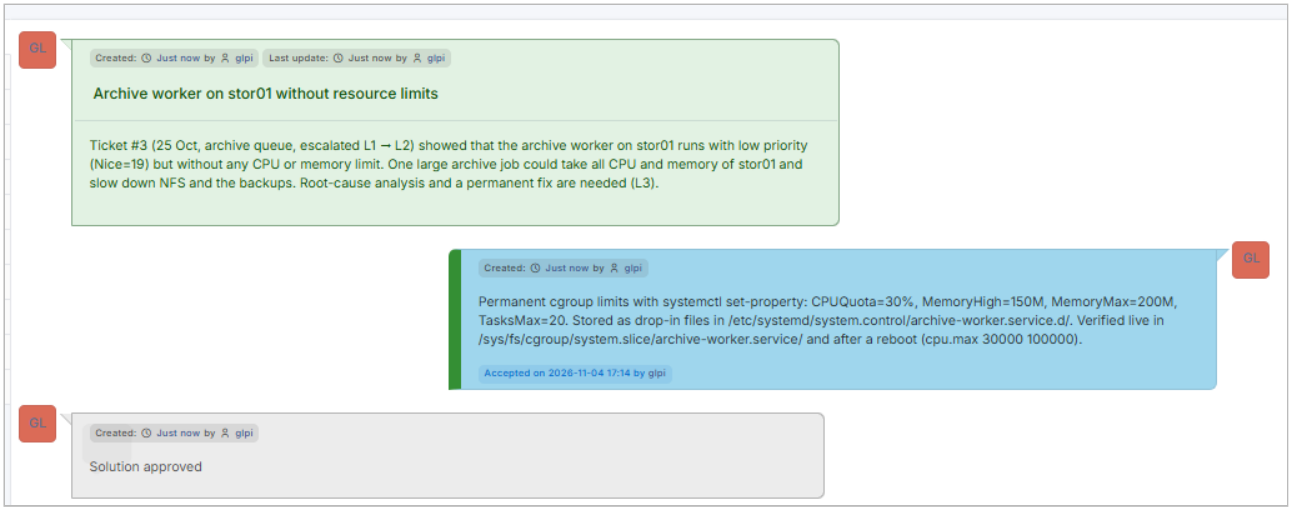
Analysis – the three boxes are part of the Problem form:





Impacts: NFS and backups could become slow · Causes: no cgroup limits, only Nice=19 · Symptoms: high CPU and memory during a big archive job.

B – Solution: the Step 4 limits – approved



The solution records exactly what was changed and how it was verified: `CPUQuota=30%`, `MemoryHigh=150M`, `MemoryMax=200M`, `TasksMax=20`, drop-in files in `/etc/systemd/system.control/archive-worker.service.d/`, checked in `/sys/fs/cgroup` and after a reboot. Ticket #3 and stor01 are linked to the Problem.

C – Snapshot "50-processes"

VMware snapshot **50-processes** on all four VMs (gw01, app01, stor01, svc01) – description *Phase 4 complete: processes, signals, limits, incident drill, L3 Problem*. A safe point to return to before Phase 5.

Final paragraph: Phase 4 ends on the service-desk side. Today's drill became Incident ticket #4: requester alice, group L2 – Linux Storage, item stor01, with the symptom (df 501M, du 56K), the diagnosis (lsof +L1, PID 5468, deleted file still open), the solution and the prevention – solved, approved and closed. Behind ticket #3 from Day 8 I opened an L3 Problem, "Archive worker on stor01 without resource limits", assigned to L3-Admin, with its analysis – impact on NFS and backups, cause no cgroup limits (only Nice=19), symptom high CPU and memory during big jobs – and the permanent solution from Step 4: CPUQuota=30%, MemoryHigh=150M, MemoryMax=200M, TasksMax=20 as drop-in files, verified after a reboot. An Incident fixes one event; a Problem removes the cause so it does not come back. A snapshot "50-processes" of all four VMs closes Phase 4.

Before Phase 5 (logging) – the 6 things from Phases 1–4 you will use again

5-minute revision · left = what you already know · right = where it comes back in Phase 5 (journald · rsyslog · logrotate)

1 systemd services — Phase 3

- every program runs as a .service unit
- systemctl status / start / restart / enable
- unit file + drop-ins in /etc/systemd/system/

→ systemd-journald and rsyslog ARE services · each service's output lands in the journal → journalctl -u nginx

2 journalctl — Phases 3–4

- journalctl -u <service> = one service's log
- journalctl -k = kernel messages (the OOM kills!)
- --since "5 min ago" · | grep -i killed · | tail -5

→ the main tool of Phase 5 – filter by service, time, priority (-p err), boot (-b), follow live (-f)

3 RAM vs disk – live vs permanent — Phase 4

- /run = RAM → gone at reboot
- /var = disk → stays
- a setting is permanent only if it is in a FILE on the disk

→ journal in /run/log/journal = lost at reboot · in /var/log/journal = kept → journald.conf Storage=persistent

4 deleted file still open — Phase 4 drill

- rm removes only the name
- a service holding its log keeps the space
- df vs du · lsof +L1 · restart the service

→ exactly why logrotate exists: rotate logs safely (copytruncate or a service reload) instead of deleting them

5 disks and space — Phases 2–3

- df -h · du -sh · LVM volumes
- disk-full alert timer from Phase 3
- /var/log can fill a disk

→ logs grow forever → logrotate (daily, rotate 7, compress) + journald limits (SystemMaxUse=)

6 network + the 4 servers — Phase 1

- gw01 .1 · app01 .10 · stor01 .20 · svc01 .40
- ports, firewall (iptables) on gw01
- SSH between the servers

→ rsyslog sends logs from app01/gw01/svc01 to stor01 = the central log server (UDP/TCP port 514)

Where Phase 5 is going – logs from every server end up on stor01



Final paragraph: Phase 5 is about logs, and it builds on six things I already know. Services (Phase 3): journald and rsyslog are services, and every service writes into the journal. journalctl (Phases 3–4): -u for one service, -k for the kernel like the OOM kills, with --since, grep and tail. RAM vs disk (Phase 4): a journal in /run is lost at reboot, one in /var/log/journal is kept – permanent only when written to the disk. The deleted-but-open file (Phase 4 drill): the reason logrotate rotates logs safely instead of deleting them. Disks and space (Phases 2–3): logs grow forever, so they need rotation, compression and size limits. Network and the four servers (Phase 1): rsyslog will send logs from gw01, app01 and svc01 over port 514 to stor01, the central log server.